43.2.3 Linux

通过包管理器安装一些额外软件包方可编译。

Debian 一家子：

```
$ sudo apt install liblua5.4-dev libsqlite3-dev libreadline-dev libssl-dev␣
↪swig
```

Arch Linux：

```
$ sudo pacman -Sy lua sqlite swig openssl libgit2
```

然后使用配置好的 QtCreator 环境即可编译。

如果你不想用 Qt 安装器的话，可以用包管理器安装依赖，下面仅举例 Arch：

```
$ sudo pacman -S qt6-base qt6-declarative qt6-5compat qt6-multimedia
$ sudo pacman -S cmake lua sqlite swig openssl swig
```

然后可以用命令行编译：

```
$ mkdir build && cd build
$ cmake ..
$ make -j8
```

如果你使用 Nix/NixOs 的话，可以在 clone repo 后直接使用 nix flake 构建：

```
$ git clone https://github.com/Notify-ctrl/FreeKill
$ nix build '.?submodules=1'
```

43.2.4 MacOS

大致与 Windows 类似，但尚且缺少确切的方案。谁 v 我台 Mac

43.2.5 编译安卓版

用 Qt 安装器装好 Android 库，然后配置一下 android-sdk 就能编译了。

Part VII

API 参考

CHAPTER 44

Engine

class Engine: Object

Engine 是整个 FreeKill 赖以运行的核心。

它包含了 FreeKill 涉及的所有武将、卡牌、游戏模式等等

同时也提供了许多常用的函数。

extensions: dict[str, list[str]]

所有 mod 列表及其包含的拓展包

extension_names: list[str]

Mod 名字的数组，为了方便排序

packages: dict[str, Package]

所有拓展包的列表

package_names: list[str]

含所有拓展包名字的数组，为了方便排序

skills: dict[str, Skill]

所有的技能

related_skills: dict[str, list[Skill]]

所有技能的关联技能

global_trigger: list[TriggerSkill]

所有的全局触发技

global_status_skill: dict[class, list[Skill]]

所有的全局状态技

generals: dict[str, General]
所有武将

same_generals: dict[str, list[str]]
所有同名武将组合

lords: list[str]
所有主公武将，用于常备主公

all_card_types: dict[str, Card]
所有的卡牌类型以及一张样板牌

cards: list[Card]
所有卡牌

translations: dict[str, dict[str, str]]
翻译表

game_modes: dict[str, GameMode]
所有游戏模式

game_mode_disabled: dict[str, list[str]]
游戏模式禁用的包

currentResponsePattern: str
要求用牌的种类（如要求用特定花色的桃 · · ·）

currentResponseReason: str
要求用牌的原因（如濒死，被特定牌指定，使用特定技能 · · ·）

filtered_cards: dict[integer, Card]
被锁视技影响的卡牌

printed_cards: dict[integer, Card]
被某些房间现场打印的卡牌，id 都是负数且从-2 开始

poxi_methods: dict[str, PoxiSpec]
“魄袭”框操作方法表

qml_marks: dict[str, QmlMarkSpec]
自定义 Qml 标记的表

mini_games: dict[str, MiniGameSpec]
自定义多人交互表

initialize()
Engine 的构造函数。

这个函数只应该被执行一次。执行了之后，会创建一个 Engine 实例，并放入全局变量 Fk 中。

Return type
nil

loadPackage(*pack*)

向 Engine 中加载一个拓展包。

会加载这个拓展包含有的所有武将、卡牌以及游戏模式。

Parameters

pack (*Package*) –要加载的拓展包

loadPackages()

加载所有拓展包。

Engine 会在 packages/下搜索所有含有 init.lua 的文件夹，并把它们作为拓展包加载进来。

这样的 init.lua 可以返回单个拓展包，也可以返回拓展包数组，或者什么都不返回。

标包和标准卡牌包比较特殊，它们永远会在第一个加载。

Return type

nil

loadDisabled()

Return type

nil

loadTranslationTable(*t*, *lang?*)

向翻译表中加载新的翻译表。

Parameters

- **t** (table) –要加载的翻译表，这是一个原文–> 译文的键值对表
- **lang?** (str) –目标语言，默认为 zh_CN

translate(*src*, *lang?*)

翻译一段文本。其实就是从翻译表中去找

Parameters

- **src** (str) –要翻译的文本
- **lang?** (str) –要使用的语言，默认读取 config

addSkill(*skill*)

向 Engine 中加载一个技能。

如果技能是 global 的，那么同时会将其放到那些 global 技能表中。

如果技能有关联技能，那么递归地加载那些关联技能。

Parameters

skill (*Skill*) –要加载的技能

addSkills(*skills*)

加载一系列技能。

Parameters

skills (`list[Skill]`) –要加载的技能数组

addGeneral(*general*)

加载一个武将到 Engine 中。

如果武将的 trueName 和 name 不同的话，那么也会将其加到同将清单中。

Parameters

general (*General*) –要添加的武将

addGenerals(*generals*)

加载一系列武将。

Parameters

generals (`list[General]`) –要加载的武将列表

canUseGeneral(*g*)

判断一个武将是否在本房间可用。

Parameters

g (`str`) –武将名

getSameGenerals(*name*)

根据武将名称，获取它的同名武将。

注意以此法返回的同名武将列表不包含他自己。

Parameters

name (`str`) –要查询的武将名字

Returns

这个武将对应的同名武将列表

Return type

`list[str]`

addCard(*card*)

向 Engine 中加载一张卡牌。

卡牌在加载的时候，会被赋予一个唯一的 id。(从 1 开始)

Parameters

card (*Card*) –要加载的卡牌

addCards(*cards*)

向 Engine 中加载一系列卡牌。

Parameters

cards (`list[Card]`) –要加载的卡牌列表

cloneCard(*name*, *suit?*, *number?*)

根据牌名、花色、点数，复制一张牌。

返回的牌是一张虚拟牌。

Parameters

- **name** (str) –牌名
- **suit?** (Suit) –花色
- **number?** (integer) –点数

Return type

Card

addGameModes(*game_modes*)

向 Engine 中添加一系列游戏模式。

Parameters

game_modes (list[GameMode]) –要添加的游戏模式列表

addGameMode(*game_mode*)

向 Engine 中添加一个游戏模式。

Parameters

game_mode (*GameMode*) –要添加的游戏模式

addGameEvent(*name*, *pfunc*, *mfunc*, *cfunc*, *efunc*)

Parameters

- **name** (any) –
- **pfunc** (any) –
- **mfunc** (any) –
- **cfunc** (any) –
- **efunc** (any) –

addPoxiMethod(*spec*)

Parameters

spec (PoxiSpec) –

addQmlMark(*spec*)

Parameters

spec (QmlMarkSpec) –

addMiniGame(*spec*)

Parameters

spec (MiniGameSpec) –

getGeneralsRandomly(*num*, *generalPool?*, *except?*, *filter?*)

从已经开启的拓展包中，随机选出若干名武将。

对于同名武将不会重复选取。

如果符合条件的武将不够，那么就不能保证能选出那么多武将。

Parameters

- **num** (integer) –要选出的武将数量
- **generalPool?** (list[General]) –选择的范围，默认是已经启用的所有武将
- **except?** (list[str]) –特别要排除掉的武将名列表，默认是空表
- **filter?** (fun(General):boolean) –? @ 可选参数，若这个函数返回 true 的话这个武将被排除在外

Returns

随机选出的武将列表

Return type

list[General]

getAllGenerals(*except?*)

获取已经启用的所有武将的列表。

Parameters

except? (list[General]) –特别指明要排除在外的武将

Returns

所有武将的列表

Return type

list[General]

getAllCardIds(*except?*)

获取当前已经启用的所有卡牌。

Parameters

except? (list[integer]) –特别指定要排除在外的 id 列表

Returns

所有卡牌 id 的列表

Return type

list[integer]

getCardById(*id*, *ignoreFilter?*)

根据 id 返回相应的卡牌。

Parameters

- **id** (integer) –牌的 id

- **ignoreFilter?** (boolean) –是否要无视掉锁定视为技，直接获得真牌

Returns
这个 id 对应的卡牌

Return type
Card

filterCard(*id*, *player*, *data*)
对那个 id 应用锁定视为技，将它变成要被锁定视为的牌。

Parameters

- **id** (integer) –要处理的 id
- **player** (*Player*) –和这张牌扯上关系的那名玩家
- **data** (any) –随意，目前只用到 JudgeStruct，为了影响判定牌

currentRoom()
获知当前的 Engine 是跑在服务端还是客户端，并返回相应的实例。

Return type
AbstractRoom

getDescription(*name*)
根据字符串获得这个技能或者这张牌的描述

其实就是翻译了“:” .. name 罢了

Parameters
name (str) –要获得描述的名字

Returns
描述

Return type
str

CHAPTER 45

Card

class Card: Object

Card 记录了 FreeKill 所有卡牌的基础信息。

它包含了 ID、所属包、牌名、花色、点数等等

id: integer

标志某一张卡牌唯一的数字，从 1 开始。若此牌是虚拟牌，则其 id 为 0。服务器启动时为卡牌赋予 ID。

package: Package

卡牌所属的扩展包

name: str

卡牌的名字

suit: Suit

卡牌的花色（四色及无花色）

number: integer

卡牌的点数（0 到 K）

trueName: str

卡牌的真名，一般用于分辨杀。

color: Color

卡牌的颜色（分为黑色、红色、无色）

type: CardType

卡牌的种类（基本牌、锦囊牌、装备牌）

sub_type: CardSubtype

卡牌的子种类（例如延时锦囊牌、武器、防具等）

area: CardArea

卡牌所在区域（例如手牌区，判定区，装备区，牌堆，弃牌堆 ···）

mark: dict[str, integer]

当前拥有的所有标记，用烂了

subcards: list[integer]

子卡 ID 表

skillName: str

虚拟牌的技能名 for virtual cards

skillNames: list[str]

虚拟牌的技能名们（一张虚拟牌可能有多个技能名，如芳魂、龙胆、朱雀羽扇）

skill: Skill

技能（用于实现卡牌效果）

special_skills: None

衍生技能，如重铸

is_damage_card: boolean

是否为会造成伤害的牌

multiple_targets: boolean

是否为指定多个目标的牌

is_passive: None

是否只能在响应时使用或打出

is_derived: None

判断是否为衍生牌

initialize(*name*, *suit*, *number*, *color*)

Card 的构造函数。具体负责构建 Card 实例的函数，请参见 fk_ex 部分。

Parameters

- **name** (any) –
- **suit** (any) –
- **number** (any) –
- **color** (any) –

clone(*suit?*, *number?*)

克隆特定卡牌并赋予花色与点数。

会将 skill/special_skills/equip_skill 继承到克隆牌中。

Parameters

- **suit?** (Suit) –克隆后的牌的花色
- **number?** (integer) –克隆后的牌的点数

Returns
产品

Return type
Card

isVirtual()
检测是否为虚拟卡牌，如果其 ID 为 0，则为虚拟卡牌。

getEffectiveId()
获取卡牌的 ID。

如果牌是虚拟牌，则返回其第一张子卡的 id，没有子卡就返回 nil

Returns

?

Return type
integer

addSubcard(*card*)
将一张子卡牌加入某张牌中（是 addSubcards 的基础函数，常用 addSubcards）。

Parameters
card (integer or Card) –要加入的子卡

addSubcards(*cards*)
将一批子卡牌加入某张牌中（常用于将这批牌弃置/交给某个角色 ···）。

Parameters
cards (list[integer] or list[Card]) –要加入的子卡列表

clearSubcards()
清空加入某张牌中的子卡牌。

matchPattern(*pattern*)
判断此牌能否符合一个卡牌规则。

Parameters
pattern (any) –

getSuitString(*symbol?*)
获取卡牌花色并返回花色文字描述（如黑桃、红桃、梅花、方块）或者符号（如 ，带颜色）。

Parameters
symbol? (boolean) –是否以符号形式显示

Returns
描述花色的字符串

Return type
`str`

getColorString()
获取卡牌颜色并返回点数颜色描述（例如黑色/红色/无色）。

Returns
描述颜色的字符串

Return type
`str`

getTypeString()
获取卡牌类型并返回类型描述（例如基本牌/锦囊牌/装备牌）。

getSubtypeString()

getNumberStr(*num*)
获取卡牌点数并返回点数文字描述（仅限 A/J/Q/K/X）。

Parameters
num (`integer`) –当你只想翻译点数为文字时 (优先检查，请注意)

strToNumber(*str*)
根据点数文字描述返回数字。

Parameters
str (`integer`) –只能翻译文字为点数

getSuitCompletedString(*symbol*)
获取卡牌的完整点数 (花色 + 点数)，如（黑桃 A/ A)。

Parameters
symbol (`boolean`) –是否以符号形式显示花色

Returns
完整点数（字符串）

Return type
`str`

isCommonTrick()
判断卡牌是否为普通锦囊牌

Return type
`boolean`

addMark(*mark*, *count*)
为卡牌赋予 Mark。

Parameters

- **mark** (str) –标记
- **count** (integer) –为标记赋予的数量

removeMark(*mark*, *count*)

为卡牌移除 Mark。

Parameters

- **mark** (str) –标记
- **count** (integer) –为标记删除的数量

setMark(*mark*, *count*)

为卡牌设置 Mark 至指定数量。

关于标记的说明：

- @ 开头的为可见标记，其余为隐藏标记。
- -turn 结尾、-phase 结尾、-round 结尾的如同玩家标记一样在这个时机自动清理。
- -noclear 结尾的表示不要自动清理。
- 默认的自动清理策略是当卡牌离开手牌区后清除所有的标记。
- -turn 之类的后缀会覆盖默认清理的方式。
- (TODO: 以上皆为画饼)

Parameters

- **mark** (str) –标记
- **count** (integer) –为标记删除的数量

getMark(*mark*)

获取卡牌对应 Mark 的数量。

Parameters

mark (str) –标记

Return type

any

getMarkNames()

获取卡牌有哪些 Mark。

compareSuitWith(*anotherCard*, *diff?*)

比较两张卡牌的花色是否相同

Parameters

- **anotherCard** (*Card*) –另一张卡牌
- **diff?** (boolean) –比较二者不同

Returns
返回比较结果

Return type
`boolean`

compareColorWith(*anotherCard*, *diff?*)

比较两张卡牌的颜色是否相同

Parameters

- **anotherCard** (*Card*) –另一张卡牌
- **diff?** (boolean) –比较二者不同

Returns
返回比较结果

Return type
`boolean`

compareNumberWith(*anotherCard*, *diff?*)

比较两张卡牌的点数是否相同

Parameters

- **anotherCard** (*Card*) –另一张卡牌
- **diff?** (boolean) –比较二者不同

Returns
返回比较结果

Return type
`boolean`

toLogString()

获取卡牌的文字信息并准备作为 log 发送。

getIdList(*c*)

静态方法。传入下列类型之一的参数，返回 id 列表。

Parameters
c (integer or list[integer] or Card or list[Card]) –

Return type
`list[integer]`

45.1 BasicCard

`class BasicCard:` *Card*

`initialize`(*name*, *suit*, *number*)

Parameters

- **name** (any) –
- **suit** (any) –
- **number** (any) –

45.2 TrickCard

`class TrickCard:` *Card*

`initialize`(*name*, *suit*, *number*)

Parameters

- **name** (any) –
- **suit** (any) –
- **number** (any) –

45.2.1 DelayedTrickCard

`class DelayedTrickCard:` *TrickCard*

`initialize`(*name*, *suit*, *number*)

Parameters

- **name** (any) –
- **suit** (any) –
- **number** (any) –

45.3 EquipCard

class EquipCard: *Card*

equip_skill: Skill

initialize(*name*, *suit*, *number*)

Parameters

- **name** (any) –
- **suit** (any) –
- **number** (any) –

onInstall(*room*, *player*)

Parameters

- **room** (*Room*) –
- **player** (*Player*) –

onUninstall(*room*, *player*)

Parameters

- **room** (*Room*) –
- **player** (*Player*) –

clone(*suit*, *number*)

Parameters

- **suit** (any) –
- **number** (any) –

45.3.1 Weapon

class Weapon: *EquipCard*

attack_range: integer

initialize(*name*, *suit*, *number*, *attackRange*)

Parameters

- **name** (any) –
- **suit** (any) –
- **number** (any) –
- **attackRange** (any) –

`clone`(*suit*, *number*)

Parameters

- **suit** (any) –
- **number** (any) –

45.3.2 Armor

`class Armor:` *EquipCard*

`initialize`(*name*, *suit*, *number*)

Parameters

- **name** (any) –
- **suit** (any) –
- **number** (any) –

45.3.3 DefensiveRide

`class DefensiveRide:` *EquipCard*

`initialize`(*name*, *suit*, *number*)

Parameters

- **name** (any) –
- **suit** (any) –
- **number** (any) –

45.3.4 OffensiveRide

`class OffensiveRide:` *EquipCard*

`initialize`(*name*, *suit*, *number*)

Parameters

- **name** (any) –
- **suit** (any) –
- **number** (any) –

45.3.5 Treasure

class Treasure: *EquipCard*

initialize(*name*, *suit*, *number*)

Parameters

- **name** (any) –
- **suit** (any) –
- **number** (any) –

CHAPTER 46

Skill

class Skill: Object

Skill 用来描述一个技能。

name: str

技能名

trueName: str

技能真名

package: Package

技能所属的包

frequency: Frequency

技能发动的频繁程度，通常 compulsory（锁定技）及 limited（限定技）用的多。

visible: boolean

技能是否会显示在游戏中

mute: boolean

决定是否关闭技能配音

global: boolean

决定是否是全局技能

anim_type: str

技能类型定义

related_skills: list[Skill]

和本技能相关的其他技能，有时候一个技能实际上是通过好几个技能拼接而实现的。

attached_equip: str

属于什么装备的技能?

relate_to_place: str

主将技/副将技

switchSkillName: str

转换技名字

initialize(*name*, *frequency*)

构造函数，不可随意调用。

Parameters

- **name** (str) –技能名
- **frequency** (Frequency) –技能发动的频繁程度，通常 compulsory（锁定技）及 limited（限定技）用的多。

addRelatedSkill(*skill*)

为一个技能增加相关技能。

Parameters

skill (*Skill*) –技能

isEquipmentSkill()

确认本技能是否为装备技能。

Return type

boolean

isEffectable(*player*)

判断技能是不是对于某玩家而言失效了。

它影响的是 hasSkill，但也可以单独拿出来判断。

Parameters

player (*Player*) –玩家

Return type

boolean

addAttachedKingdom(*kingdom*)

为技能增加所属势力，需要在隶属特定势力时才能使用此技能。

案例：手杀文鸯

Parameters

kingdom (any) –

isSwitchSkill()

判断某个技能是否为转换技

46.1 UsableSkill

class UsableSkill: *Skill*

main_skill: UsableSkill

max_use_time: list[integer]

expand_pile: None

?

derived_piles: None

initialize(*name*, *frequency*)

Parameters

- **name** (any) –
- **frequency** (any) –

getMaxUseTime(*player*, *scope*, *card*, *to*)

Parameters

- **player** (any) –
- **scope** (any) –
- **card** (any) –
- **to** (any) –

withinTimesLimit(*player*, *scope*, *card*, *card_name*, *to*)

Parameters

- **player** (any) –
- **scope** (any) –
- **card** (any) –
- **card_name** (any) –
- **to** (any) –

46.1.1 ActiveSkill

class ActiveSkill: *UsableSkill*

min_target_num: integer

max_target_num: integer

target_num: integer

target_num_table: list[integer]

min_card_num: integer

max_card_num: integer

card_num: integer

card_num_table: list[integer]

interaction: any

initialize(*name*, *frequency*)

Parameters

- **name** (any) –
- **frequency** (any) –

canUse(*player*, *card*)

{ Determine whether the skill can be used in playing phase

Parameters

- **player** (*Player*) –
- **card** (*Card*) –helper

cardFilter(*to_select*, *selected*, *selected_targets*)

Determine whether a card can be selected by this skill

only used in skill of players

Parameters

- **to_select** (integer) –id of a card not selected
- **selected** (list[integer]) –ids of selected cards
- **selected_targets** (list[integer]) –ids of selected players

targetFilter(*to_select*, *selected*, *selected_cards*, *card*, *extra_data?*)

Determine whether a target can be selected by this skill

only used in skill of players

Parameters

- **to_select** (integer) –id of the target
- **selected** (list[integer]) –ids of selected targets
- **selected_cards** (list[integer]) –ids of selected cards
- **card** (*Card*) –helper
- **extra_data?** (any) –extra_data

modTargetFilter(*to_select*, *selected?*, *user?*, *card?*, *distance_limited*)

Determine whether a target can be selected by this skill(in modifying targets)

only used in skill of players

Parameters

- **to_select** (integer) –id of the target
- **selected?** (list[integer]) –ids of selected targets
- **user?** (integer) –id of the userdata
- **card?** (*Card*) –helper
- **distance_limited** (boolean) –is limited by distance

getMinTargetNum()

getMaxTargetNum(*player*, *card*)

Parameters

- **player** (any) –
- **card** (any) –

getMinCardNum()

getMaxCardNum()

getDistanceLimit(*player*, *card*, *to*)

Parameters

- **player** (any) –
- **card** (any) –
- **to** (any) –

withinDistanceLimit(*player*, *isattack*, *card*, *to*)

Parameters

- **player** (any) –
- **isattack** (any) –
- **card** (any) –
- **to** (any) –

feasible(*selected*, *selected_cards*)

Determine if selected cards and targets are valid for this skill

If returns true, the OK button should be enabled only used in skill of players

Parameters

- **selected** (list[integer]) –ids of selected players

- **selected_cards** (list[integer]) –ids of selected cards

onUse(*room*, *cardUseEvent*)

}

Parameters

- **room** (*Room*) –
- **cardUseEvent** (*CardUseStruct*) –

onAction(*room*, *cardUseEvent*, *isEnding?*)

Parameters

- **room** (*Room*) –
- **cardUseEvent** (*CardUseStruct*) –
- **isEnding?** (boolean) –

aboutToEffect(*room*, *cardEffectEvent*)

Parameters

- **room** (*Room*) –
- **cardEffectEvent** (CardEffectEvent or SkillEffectEvent) –

onEffect(*room*, *cardEffectEvent*)

Parameters

- **room** (*Room*) –
- **cardEffectEvent** (CardEffectEvent or SkillEffectEvent) –

onNullified(*room*, *cardEffectEvent*)

Parameters

- **room** (*Room*) –
- **cardEffectEvent** (CardEffectEvent or SkillEffectEvent) –

prompt(*selected_cards*, *selected_targets*)

Parameters

- **selected_cards** (list[integer]) –ids of selected cards
- **selected_targets** (list[integer]) –ids of selected players

46.1.2 ViewAsSkill

class ViewAsSkill: *UsableSkill*

pattern: str

cards that can be viewAs' ed by this skill

interaction: any

initialize(*name*, *frequency*)

Parameters

- **name** (any) –
- **frequency** (any) –

cardFilter(*to_select*, *selected*)

Parameters

- **to_select** (integer) –id of a card not selected
- **selected** (list[integer]) –ids of selected cards

Return type

boolean

viewAs(*cards*)

Parameters

cards (list[integer]) –ids of cards

Returns

?

Return type

Card

enabledAtPlay(*player*)

Parameters

player (*Player*) –

enabledAtResponse(*player*)

Parameters

player (*Player*) –

beforeUse(*player*, *cardUseStruct*)

Parameters

- **player** (*Player*) –
- **cardUseStruct** (*CardUseStruct*) –

afterUse(*player*, *cardUseStruct*)

Parameters

- **player** (*Player*) –
- **cardUseStruct** (*CardUseStruct*) –

prompt(*selected_cards*, *selected_targets*)

Parameters

- **selected_cards** (`list[integer]`) –ids of selected cards
- **selected_targets** (`list[integer]`) –ids of selected players

46.1.3 TriggerSkill

class TriggerSkill: *UsableSkill*

global: boolean

events: list[Event]

refresh_events: list[Event]

priority_table: dict[Event, number]

initialize(*name*, *frequency*)

Parameters

- **name** (`any`) –
- **frequency** (`any`) –

canRefresh(*event*, *target*, *player*, *data*)

Determine whether a skill can refresh at this moment

Parameters

- **event** (`Event`) –TriggerEvent
- **target** (*ServerPlayer*) –Player who triggered this event
- **player** (*ServerPlayer*) –Player who is operating
- **data** (`any`) –useful data of the event

refresh(*event*, *target*, *player*, *data*)

Refresh the skill (e.g. clear marks)

Parameters

- **event** (`Event`) –TriggerEvent
- **target** (*ServerPlayer*) –Player who triggered this event

- **player** (*ServerPlayer*) –Player who is operating
- **data** (any) –useful data of the event

triggerable(*event*, *target*, *player*, *data*)

Determine whether a skill can trigger at this moment

Parameters

- **event** (Event) –TriggerEvent
- **target** (*ServerPlayer*) –Player who triggered this event
- **player** (*ServerPlayer*) –Player who is operating
- **data** (any) –useful data of the event

Return type

boolean

trigger(*event*, *target*, *player*, *data*)

Parameters

- **event** (Event) –TriggerEvent
- **target** (*ServerPlayer*) –Player who triggered this event
- **player** (*ServerPlayer*) –Player who is operating
- **data** (any) –useful data of the event

Returns

returns true if trigger is broken

Return type

boolean

doCost(*event*, *target*, *player*, *data*)

Parameters

- **event** (any) –
- **target** (any) –
- **player** (any) –
- **data** (any) –

cost(*event*, *target*, *player*, *data*)

Parameters

- **event** (Event) –TriggerEvent
- **target** (*ServerPlayer*) –Player who triggered this event
- **player** (*ServerPlayer*) –Player who is operating
- **data** (any) –useful data of the event

Returns
returns true if trigger is broken

Return type
`boolean`

`use(`*event*, *target*, *player*, *data*`)`

Use this skill

Parameters

- **event** (`Event`) –TriggerEvent
- **target** (*ServerPlayer*) –Player who triggered this event
- **player** (*ServerPlayer*) –Player who is operating
- **data** (`any`) –useful data of the event

Returns

?

Return type
`boolean`

`canWake(`*event*, *target*, *player*, *data*`)`

Parameters

- **event** (`any`) –
- **target** (`any`) –
- **player** (`any`) –
- **data** (`any`) –

`enableToWake(`*event*, *target*, *player*, *data*`)`

Parameters

- **event** (`Event`) –TriggerEvent
- **target** (*ServerPlayer*) –Player who triggered this event
- **player** (*ServerPlayer*) –Player who is operating
- **data** (`any`) –useful data of the event

Return type
`boolean`

46.2 StatusSkill

`class StatusSkill:` *Skill*

`global:` boolean

`initialize`(*name*, *frequency*)

Parameters

- `name` (any) –
- `frequency` (any) –

46.2.1 DistanceSkill

`class DistanceSkill:` *StatusSkill*

`getCorrect`(*from*, *to*)

Parameters

- `from` (*Player*) –
- `to` (*Player*) –

Return type
`integer`

`getFixed`(*from*, *to*)

Parameters

- `from` (*Player*) –
- `to` (*Player*) –

Return type
`integer or nil`

46.2.2 AttackRangeSkill

`class AttackRangeSkill:` *StatusSkill*

`getCorrect`(*from*)

Parameters
`from` (*Player*) –

Return type
`integer`

`getFixed`(*from*)

Parameters
`from` (*Player*) –

Return type
`integer or nil`

`withinAttackRange`(*from*, *to*)

Parameters

- `from` (`any`) –
- `to` (`any`) –

46.2.3 MaxCardsSkill

`class MaxCardsSkill:` *StatusSkill*

`getFixed()`

Returns

?

Return type
`integer`

`getCorrect()`

Return type
`integer`

`excludeFrom`(*card*)

Parameters
`card` (*Card*) –

Return type
`boolean`

46.2.4 ProhibitSkill

`class ProhibitSkill:` *StatusSkill*

`isProhibited`(*from*, *to*, *card*)

Parameters

- `from` (*Player*) –
- `to` (*Player*) –

- **card** (*Card*) –

Return type
boolean

prohibitUse(*player*, *card*)

Parameters

- **player** (*Player*) –
- **card** (*Card*) –

Return type
boolean

prohibitResponse(*player*, *card*)

Parameters

- **player** (*Player*) –
- **card** (*Card*) –

Return type
boolean

prohibitDiscard(*player*, *card*)

Parameters

- **player** (*Player*) –
- **card** (*Card*) –

Return type
boolean

prohibitPindian(*from*, *to*)

Parameters

- **from** (*Player*) –
- **to** (*Player*) –

Return type
boolean

46.2.5 InvaliditySkill

`class InvaliditySkill:` *StatusSkill*

`getInvalidity`(*from*, *skill*)

Parameters

- **from** (*Player*) –
- **skill** (*Skill*) –

Return type
boolean

46.2.6 FilterSkill

`class FilterSkill:` *StatusSkill*

`cardFilter`(*card*, *player*, *isJudgeEvent*)

Parameters

- **card** (*Card*) –
- **player** (*Player*) –
- **isJudgeEvent** (boolean) –

`viewAs`(*card*, *player*)

Parameters

- **card** (*Card*) –
- **player** (*Player*) –

Return type
Card

46.2.7 TargetModSkill

`class TargetModSkill:` *StatusSkill*

`bypassTimesCheck`(*player*, *card_skill*, *scope*, *card*)

Parameters

- **player** (*Player*) –
- **card_skill** (*ActiveSkill*) –
- **scope** (integer) –
- **card** (*Card*) –

`getResidueNum`(*player*, *card_skill*, *scope*, *card*)

Parameters

- **player** (*Player*) –
- **card_skill** (*ActiveSkill*) –
- **scope** (integer) –
- **card** (*Card*) –

`bypassDistancesCheck`(*player*, *card_skill*, *card*)

Parameters

- **player** (*Player*) –
- **card_skill** (*ActiveSkill*) –
- **card** (*Card*) –

`getDistanceLimit`(*player*, *card_skill*, *card*)

Parameters

- **player** (*Player*) –
- **card_skill** (*ActiveSkill*) –
- **card** (*Card*) –

`getExtraTargetNum`(*player*, *card_skill*, *card*)

Parameters

- **player** (*Player*) –
- **card_skill** (*ActiveSkill*) –
- **card** (*Card*) –

CHAPTER 47

General

class General: Object

General 用来描述一个武将。

所谓武将，就是所属包、武将基础信息的集合。

也许 ··· 类似于身份证?

package: Package

武将所属包

name: str

武将名字

trueName: str

武将真名，也许可以分辨标界?

kingdom: str

武将所属势力

subkingdom: str

武将副势力

hp: integer

武将初始体力

maxHp: integer

武将初始最大体力

mainMaxHpAdjustedValue: integer

主将体力上限调整

deputyMaxHpAdjustedValue: integer

副将体力上限调整

shield: integer

初始护甲

gender: Gender

武将性别

skills: list[Skill]

武将技能

other_skills: list[str]

武将身上属于其他武将的技能，通过字符串调用

related_skills: list[Skill]

武将相关的不属于其他武将的技能，例如邓艾的急袭

related_other_skills: list[str]

武将相关的属于其他武将的技能，例如孙策的英姿

companions: list[str]

有珠联璧合关系的武将

hidden: boolean

不在选将框里出现，可以点将，可以在武将一览里查询到

total_hidden: boolean

完全隐藏

initialize(*package*, *name*, *kingdom*, *hp*, *maxHp*, *gender*)

构造函数，不可随意调用。

Parameters

- **package** (*Package*) –武将所属包
- **name** (str) –武将名字
- **kingdom** (str) –武将所属势力
- **hp** (integer) –武将初始体力
- **maxHp** (integer) –武将初始最大体力
- **gender** (Gender) –武将性别

addSkill(*skill*)

为武将增加技能，需要注意增加其他武将技能时的处理方式。

Parameters

skill (Skill or str) –（单个）武将技能

addRelatedSkill(*skill*)

为武将增加相关技能，需要注意增加其他武将技能时的处理方式。

Parameters

skill (Skill or str) –（单个）武将技能

getSkillNameList(*include_lord?*)

获取武将所有技能。

Parameters

include_lord? (boolean) –

Return type

list[str]

addCompanions(*name*)

为武将增加珠联璧合关系武将（1 个或多个），只需写 trueName。

Parameters

name (list[str]) –武将真名（表）

CHAPTER 48

Package

class Package: Object

Package 用来描述一个 FreeKill 拓展包。

所谓拓展包，就是武将/卡牌/游戏模式的一个集合而已。

name: str

拓展包的名字

extensionName: str

拓展包对应的 mod 的名字。**‘详情... <extension name_>‘_**

type: PackageType

拓展包的类别，只会影响到选择拓展包的界面

generals: list[General]

拓展包包含的所有武将的列表

extra_skills: list[Skill]

拓展包包含的额外技能，即不属于武将的技能

related_skills: dict[str, str]

对于额外技能而言的关联技能

cards: list[Card]

拓展包包含的卡牌

game_modes: list[GameMode]

拓展包包含的游戏模式

game_modes_whitelist: None

拓展包关于游戏模式的白名单

game_modes_blacklist: None

拓展包关于游戏模式的黑名单

initialize(*name*, *_type?*)

拓展包的构造函数。

Parameters

- **name** (str) –包的名字
- **_type?** (integer) –包的类型，默认为武将包

getSkills()

获得这个包涉及的所有技能。

这也就是说，所有的武将技能再加上和武将无关的技能。

Return type

list[Skill]

addGeneral(*general*)

向拓展包中添加武将。

Parameters

general (*General*) –要添加的武将

addCard(*card*)

向拓展包中添加卡牌。

Parameters

card (*Card*) –要添加的卡牌

addCards(*cards*)

向拓展包中一次添加许多牌。

Parameters

cards (list[Card]) –要添加的卡牌的数组

addGameMode(*game_mode*)

向拓展包中添加游戏模式。

Parameters

game_mode (*GameMode*) –要添加的游戏模式。

setGameModeFilter(*whitelist*, *blacklist*)

向拓展包中设置游戏模式过滤。

Parameters

- **whitelist** (list[str]) –白名单
- **blacklist** (list[str]) –黑名单

addGameModeFilter(*whitelist*, *blacklist*)

向拓展包中添加游戏模式过滤。

Parameters

- **whitelist** (list[str]) –白名单
- **blacklist** (list[str]) –黑名单

CHAPTER 49

AbstractRoom

class AbstractRoom: Object

alive_players: list[Player]
所有存活玩家的数组

observers: list[Player]
看戏的

current: Player
当前行动者

status_skills: dict[class, list[Skill]]
这个房间中含有的状态技列表

filtered_cards: dict[integer, Card]
见于 Engine，其实在这

printed_cards: dict[integer, Card]
同上

skill_costs: dict[str, any]
用来存 skill.cost_data

card_marks: dict[integer, any]
用来存实体卡的 card.mark

banners: dict[str, any]
全局 mark

initialize()

getPlayerById(*id*)

Parameters

id (integer) –

Returns

?

Return type

Player

getCardArea(*cardId*)

获取一张牌所处的区域。

Parameters

cardId (integer or Card) –要获得区域的那张牌，可以是 Card 或者一个 id

Returns

这张牌的区域

Return type

CardArea

setBanner(*name*, *value*)

Parameters

- **name** (any) –
- **value** (any) –

getBanner(*name*)

Parameters

name (any) –

CHAPTER 50

Player

class Player: Object

玩家分为客户端要处理的玩家，以及服务端处理的玩家两种。

客户端能知道的玩家的信息十分有限，而服务端知道一名玩家的所有细节。

Player 类就是这两种玩家的基类，包含它们共用的部分。

id: integer

玩家的 id，每名玩家的 id 是唯一的。机器人的 id 是负数。

hp: integer

体力值

maxHp: integer

体力上限

shield: integer

护甲数

kingdom: str

势力

role: str

身份

general: str

武将

deputyGeneral: str

副将

gender: integer
性别

seat: integer
座位号

next: Player
下家

phase: Phase
当前阶段

faceup: boolean
是否正面朝上

chained: boolean
是否被横直

dying: boolean
是否处于濒死

dead: boolean
是否死亡

player_skills: list[Skill]
当前拥有的所有技能

derivative_skills: dict[Skill, list[Skill]]
当前拥有的派生技能

flag: list[str]
当前拥有的 flag，不过好像没用过

tag: dict[str, any]
当前拥有的所有 tag，好像也没用过

mark: dict[str, integer]
当前拥有的所有标记，用烂了

player_cards: dict[integer, list[integer]]
当前拥有的所有牌，键是区域，值是 id 列表

virtual_equips: list[Card]
当前的虚拟装备牌，其实也包含着虚拟延时锦囊这种

special_cards: dict[str, list[integer]]
类似“屯田”这种的私人牌堆

cardUsedHistory: dict[str, list[integer]]
用牌次数历史记录

skillUsedHistory: dict[str, list[integer]]
发动技能次数的历史记录

fixedDistance: dict[Player, integer]

与其他玩家的固定距离列表

buddy_list: list[integer]

队友列表，或者说自己可以观看别人手牌的那些玩家的列表

initialize()

构造函数。总之这不是随便调用的函数

setGeneral(*general*, *setHp?*, *addSkills?*)

设置角色、体力、技能。

Parameters

- **general** (*General*) –角色类型
- **setHp?** (boolean) –是否设置体力
- **addSkills?** (boolean) –是否增加技能

getGeneralMaxHp()

hasFlag(*flag*)

查询角色是否存在 flag。

Parameters

flag (str) –一种标记

setFlag(*flag*)

为角色赋予 flag。

Parameters

flag (str) –一种标记

clearFlags()

清除角色 flag。

addMark(*mark*, *count*)

为角色赋予 Mark。

Parameters

- **mark** (str) –标记
- **count** (integer) –为标记赋予的数量

removeMark(*mark*, *count*)

为角色移除 Mark。

Parameters

- **mark** (str) –标记
- **count** (integer) –为标记删除的数量

setMark(*mark*, *count?*)

为角色设置 Mark 至指定数量。

Parameters

- **mark** (str) –标记
- **count?** (integer) –为标记删除的数量

getMark(*mark*)

获取角色对应 Mark 的数量。

Parameters

mark (str) –标记

Return type

any

getMarkNames()

获取角色有哪些 Mark。

addCards(*playerArea*, *cardIds*, *specialName?*)

将指定数量的牌加入玩家的对应区域。

Parameters

- **playerArea** (PlayerCardArea) –玩家牌所在的区域
- **cardIds** (list[integer]) –牌的 ID，返回唯一牌
- **specialName?** (str) –私人牌堆名

removeCards(*playerArea*, *cardIds*, *specialName?*)

将指定数量的牌移除出玩家的对应区域。

Parameters

- **playerArea** (PlayerCardArea) –玩家牌所在的区域
- **cardIds** (list[integer]) –牌的 ID，返回唯一牌
- **specialName?** (str) –私人牌堆名

addVirtualEquip(*card*)

为玩家提供虚拟装备。

Parameters

card (*Card*) –卡牌

removeVirtualEquip(*cid*)

为玩家移除虚拟装备。

Parameters

cid (integer) –卡牌 ID，用来定位装备

getVirualEquip(*cid*)

确认玩家是否存在虚拟装备。

Parameters
cid (integer) –卡牌 ID，用来定位装备

hasDelayedTrick(*card_name*)

确认玩家判定区是否存在延迟锦囊牌。

Parameters
card_name (any) –

getCardIds(*playerAreas?*, *specialName?*)

获取玩家特定区域所有牌的 ID。

Parameters

- **playerAreas?** (PlayerCardArea or list[PlayerCardArea] or str) –玩家牌所在的区域
- **specialName?** (str) –私人牌堆名

Returns
返回对应区域的所有牌对应的 ID

Return type
list[integer]

getPile(*name*)

通过名字检索获取玩家对应的私人牌堆。

Parameters
name (str) –私人牌堆名

getPileNameOfId(*id*)

通过 ID 检索获取玩家对应的私人牌堆。

Parameters
id (integer) –私人牌堆 ID

Returns

?

Return type
str

getHandlyIds(*include_hand?*)

返回所有“如手牌般使用或打出”的牌。

或者说，返回所有名字以“&”结尾的 pile 的牌。

Parameters
include_hand? (boolean) –是否包含真正的手牌

Return type
list[integer]

getHandcardNum()

filterHandcards()

getEquipment(*cardSubtype*)

检索玩家装备区是否存在对应类型的装备。

Parameters

cardSubtype (CardSubtype) –卡牌子类

Returns

? @ 返回卡牌 ID 或 nil

Return type

integer

getEquipments(*cardSubtype*)

检索玩家装备区是否存在对应类型的装备列表。

Parameters

cardSubtype (CardSubtype) –卡牌子类

Returns

返回卡牌 ID 或空表

Return type

list[integer]

getMaxCards()

获取玩家手牌上限。

getAttackRange()

获取玩家攻击范围。

isRemoved()

获取角色是否被移除。

setFixedDistance(*other*, *num*)

修改玩家与其他角色的固定距离。

Parameters

- **other** (*Player*) –其他玩家
- **num** (integer) –距离数

removeFixedDistance(*other*)

移除玩家与其他角色的固定距离。

Parameters

other (*Player*) –其他玩家

distanceTo(*other*, *mode?*, *ignore_dead?*)

获取玩家与其他角色的实际距离。

通过二者位次 + 距离技能之和与两者间固定距离进行对比，更大的为实际距离。

Parameters

- **other** (*Player*) –其他玩家
- **mode?** (str) –计算模式 (left/right/both)
- **ignore_dead?** (boolean) –是否忽略尸体

inMyAttackRange(*other*, *fixLimit?*)

获取其他玩家是否在玩家的攻击范围内。

Parameters

- **other** (*Player*) –其他玩家
- **fixLimit?** (integer) –卡牌距离限制增加专用

getNextAlive(*ignoreRemoved?*, *num?*)

获取下家。

Parameters

- **ignoreRemoved?** (boolean) –忽略被移除
- **num?** (integer) –第几个，默认 1

Return type

ServerPlayer

getLastAlive(*ignoreRemoved*, *num?*)

获取上家。

Parameters

- **ignoreRemoved** (boolean) –忽略被移除
- **num?** (integer) –第几个，默认 1

Return type

ServerPlayer

addCardUseHistory(*cardName*, *num?*)

增加玩家使用特定牌的历史次数。

Parameters

- **cardName** (str) –牌名
- **num?** (integer) –次数

setCardUseHistory(*cardName*, *num*, *scope?*)

设定玩家使用特定牌的历史次数。

Parameters

- **cardName** (str) –牌名
- **num** (integer) –次数
- **scope?** (integer) –历史范围全为 nil 意为清空

addSkillUseHistory(*skill_name*, *num?*)

增加玩家使用特定技能的历史次数。

Parameters

- **skill_name** (str) –技能名
- **num?** (integer) –次数默认 1

setSkillUseHistory(*skill_name*, *num?*, *scope?*)

设定玩家使用特定技能的历史次数。

Parameters

- **skill_name** (str) –技能名
- **num?** (integer) –次数默认 0
- **scope?** (integer) –查询历史范围

usedCardTimes(*cardName*, *scope?*)

获取玩家使用特定牌的历史次数。

Parameters

- **cardName** (str) –牌名
- **scope?** (integer) –查询历史范围，默认 Turn

usedSkillTimes(*skill_name*, *scope?*)

获取玩家使用特定技能的历史次数。

Parameters

- **skill_name** (str) –技能名
- **scope?** (integer) –查询历史范围，默认 Turn

isKongcheng()

获取玩家是否无手牌。

isNude()

获取玩家是否无手牌及装备牌。

isAllNude()

获取玩家所有区域是否无牌。

isWounded()

获取玩家是否受伤。

getLostHp()

获取玩家已失去体力。

hasSkill(*skill*, *ignoreNullified?*, *ignoreAlive?*)

检索玩家是否有对应技能。

Parameters

- **skill** (str or Skill) –技能名
- **ignoreNullified?** (boolean) –忽略技能是否被无效
- **ignoreAlive?** (boolean) –忽略角色在场与否

addSkill(*skill*, *source_skill?*)

为玩家增加对应技能。

Parameters

- **skill** (str or Skill) –技能名
- **source_skill?** (str or Skill) –本有技能（和衍生技能相对）

Returns

got skills that Player didn’t have at start

Return type

list[Skill]

loseSkill(*skill*, *source_skill?*)

为玩家删除对应技能。

Parameters

- **skill** (str or Skill) –技能名
- **source_skill?** (str or Skill) –本有技能（和衍生技能相对）

Returns

lost skills that the Player doesn’t have anymore

Return type

list[Skill]

getAllSkills()

获取对应玩家所有技能。

canUse(*card*, *extra_data?*)

确认玩家是否可以使用特定牌。

Parameters

- **card** (*Card*) –特定牌
- **extra_data?** (*UseExtraData*) –额外数据

canUseTo(*card*, *to*, *extra_data?*)

确认玩家是否可以对特定玩家使用特定牌。

Parameters

- **card** (*Card*) –特定牌
- **to** (*Player*) –特定玩家

- **extra_data?** (*UseExtraData*) –额外数据

isProhibited(*to*, *card*)

确认玩家是否被禁止对特定玩家使用特定牌。

Parameters

- **to** (*Player*) –特定玩家
- **card** (*Card*) –特定牌

prohibitUse(*card*)

确认玩家是否被禁止使用特定牌。

Parameters

card (*Card*) –特定的牌

prohibitResponse(*card*)

确认玩家是否被禁止打出特定牌。

Parameters

card (*Card*) –特定的牌

prohibitDiscard(*card*)

确认玩家是否被禁止弃置特定牌。

Parameters

card (Card or integer) –特定的牌

prohibitReveal(*isDeputy*)

确认角色是否被禁止亮将。

Parameters

isDeputy (any) –

canPindian(*to*, *ignoreFromKong?*, *ignoreToKong?*)

Parameters

- **to** (*Player*) –
- **ignoreFromKong?** (boolean) –
- **ignoreToKong?** (boolean) –

canMoveCardIntoEquip(*cardId*, *convert?*)

判断一张牌能否移动至某角色的装备区

Parameters

- **cardId** (integer) –移动的牌
- **convert?** (boolean) –是否可以替换装备（默认可以）

Return type

boolean

getSwitchSkillState(*skillName*, *afterUse?*, *inWord?*)

获取转换技状态

Parameters

- **skillName** (str) –技能名
- **afterUse?** (boolean) –是否提前计算转换后状态
- **inWord?** (boolean) –是否返回文字

Returns

转换技状态

Return type

number or str

canMoveCardInBoardTo(*to*, *id*)

Parameters

- **to** (any) –
- **id** (any) –

canMoveCardsInBoardTo(*to*, *flag*, *excludeIds*)

Parameters

- **to** (any) –
- **flag** (any) –
- **excludeIds** (any) –

getQuestSkillState(*skillName*)

Parameters

skillName (any) –

getAvailableEquipSlots(*subtype*)

Parameters

subtype (any) –

hasEmptyEquipSlot(*subtype*)

Parameters

subtype (any) –

addBuddy(*other*)

Parameters

other (any) –

removeBuddy(*other*)

Parameters

other (any) –

`isBuddy`(*other*)

Parameters

`other` (`any`) –

`compareGenderWith`(*other*, *diff?*)

比较两名角色的性别是否相同。

Parameters

- `other` (*`Player`*) –另一名角色
- `diff?` (`boolean`) –比较二者不同

Returns

返回比较结果

Return type

`boolean`

CHAPTER 51

GameMode

class GameMode: Object

GameMode 用来描述一个游戏模式。

可以参考欢乐斗地主。

name: str

游戏模式名

minPlayer: integer

最小玩家数

maxPlayer: integer

最大玩家数

rule: None

规则（通过技能完成，通常用来为特定角色及特定时机提供触发事件）

logic: None

逻辑（通过 function 完成，通常用来初始化、分配身份及座次）

whitelist: None

白名单

blacklist: None

黑名单

initialize(*name*, *min*, *max*)

构造函数，不可随意调用。

Parameters

- **name** (str) –游戏模式名

- **min** (integer) –最小玩家数
- **max** (integer) –最大玩家数

getWinner(*victim*)

Parameters
victim (*ServerPlayer*) –死者

Returns
胜者阵营

Return type
str

surrenderFunc(*playedTime*)

Parameters
playedTime (number) –游戏时长（单位：秒）

Return type
table

countInFunc(*room*)

Parameters
room (*Room*) –游戏房间

Return type
boolean

CHAPTER 52

Exppattern

class Exppattern: Object

matchers: list[Matcher]

initialize(*spec*)

Parameters
spec (any) –

Parse(*pattern*)

Parameters
pattern (str) –

Return type
Exppattern

match(*card*)

Parameters
card (*Card*) –

matchExp(*exp*)

Parameters
exp (any) –

CHAPTER 53

Room

class Room: *AbstractRoom*

Room 是 fk 游戏逻辑运行的主要场所，同时也提供了许多 API 函数供编写技能使用。

一个房间中只有一个 Room 实例，保存在 RoomInstance 全局变量中。

room: fk.Room

C++ 层面的 Room 类实例，别管他就是了，用不着

id: integer

房间的 id

players: list[ServerPlayer]

这个房间中所有参战玩家

alive_players: list[ServerPlayer]

所有还活着的玩家

observers: list[fk.ServerPlayer]

旁观者清单，这是 c++ 玩家列表，别乱动

current: ServerPlayer

当前回合玩家

game_started: boolean

游戏是否已经开始

game_finished: boolean

游戏是否已经结束

timeout: integer

出牌时长上限

tag: dict[str, any]

Tag 清单，其实跟 Player 的标记是差不多的东西

general_pile: list[str]

武将牌堆，这是可用武将名的数组

draw_pile: list[integer]

摸牌堆，这是卡牌 id 的数组

discard_pile: list[integer]

弃牌堆，也是卡牌 id 的数组

processing_area: list[integer]

处理区，依然是卡牌 id 数组

void: list[integer]

从游戏中除外区，一样的是卡牌 id 数组

card_place: dict[integer, CardArea]

每个卡牌的 id 对应的区域，一张表

owner_map: dict[integer, integer]

每个卡牌 id 对应的主人，表的值是那个玩家的 id，可能是 nil

settings: table

房间的额外设置，差不多是 json 对象

logic: GameLogic

这个房间使用的游戏逻辑，可能根据游戏模式而变动

request_queue: dict[userdata, table]

request_self: dict[integer, integer]

skill_costs: dict[str, any]

存放 skill.cost_data 用

card_marks: dict[integer, any]

存放 card.mark 之用

initialize(*_room*)

构造函数。别去构造

Parameters

_room (fk.Room) –

resume()

makeGeneralPile()

isReady()

checkNoHuman(*chkOnly*)

Parameters
chkOnly (any) –

run()

正式在这个房间中开始游戏。

当这个函数返回之后，整个 Room 线程也宣告结束。

Return type
nil

setCardArea(*cardId*, *cardArea*, *owner?*)

基本算是私有函数，别去用

Parameters

- **cardId** (integer) –
- **cardArea** (CardArea) –
- **owner?** (integer) –

getCardArea(*cardId*)

获取一张牌所处的区域。

Parameters
cardId (integer or Card) –要获得区域的那张牌，可以是 Card 或者一个 id

Returns
这张牌的区域

Return type
CardArea

getCardOwner(*cardId*)

获得拥有某一张牌的玩家。

Parameters
cardId (integer or Card) –要获得主人的那张牌，可以是 Card 实例或者 id

Returns
? @ 这张牌的主人，可能返回 nil

Return type
ServerPlayer

getPlayerById(*id*)

根据玩家 id，获得那名玩家本人。

Parameters
id (integer) –玩家的 id

Returns
这个 id 对应的 ServerPlayer 实例

Return type
ServerPlayer

`sortPlayersByAction`(*playerIds*)
将房间中的玩家按照行动顺序重新排序。

Parameters
playerIds (list[integer]) –玩家 id 列表，这个数组会被这个函数排序

`deadPlayerFilter`(*playerIds*)

Parameters
playerIds (any) –

`getAllPlayers`(*sortBySeat?*)
获得当前房间中的所有玩家。

返回的数组的第一个元素是当前回合玩家，并且按行动顺序进行排序。

Parameters
sortBySeat? (boolean) –是否无视按座位排序直接返回

Returns
房间中玩家的数组

Return type
list[ServerPlayer]

`getAlivePlayers`(*sortBySeat?*)
获得所有存活玩家，参看 getAllPlayers

Parameters
sortBySeat? (boolean) –

Return type
list[ServerPlayer]

`getOtherPlayers`(*player*, *sortBySeat?*, *include_dead?*)
获得除一名玩家外的其他玩家。

Parameters

- **player** (*ServerPlayer*) –要排除的玩家
- **sortBySeat?** (boolean) –是否要按座位排序?
- **include_dead?** (boolean) –是否要把死人也算进去?

Returns
其他玩家列表

Return type
list[ServerPlayer]

getLord()

获得当前房间中的主公。

由于某些游戏模式没有主公，该函数可能返回 nil。

Returns

? @ 主公

Return type

ServerPlayer

getNCards(*num*, *from?*)

从摸牌堆中获取若干张牌。

注意了，这个函数会对牌堆进行实际操作，也就是说它返回一系列 id 后，牌堆中就会少这么多 id。

如果牌堆中没有足够的牌可以获得，那么会触发洗牌；还是不够的话，游戏就平局。

Parameters

- **num** (integer) –要获得的牌的数量
- **from?** (str) –获得牌的位置，可以是 "top" 或者 "bottom"，表示牌堆顶还是牌堆底

Returns

得到的 id

Return type

list[integer]

setPlayerMark(*player*, *mark*, *value*)

将一名玩家的某种标记数量相应的值。

在设置之后，会通知所有客户端也更新一下标记的值。之后的两个相同

Parameters

- **player** (*ServerPlayer*) –要被更新标记的那个玩家
- **mark** (str) –标记的名称
- **value** (any) –要设为的值，其实也可以设为字符串

addPlayerMark(*player*, *mark*, *count?*)

将一名玩家的 mark 标记增加 count 个。

Parameters

- **player** (*ServerPlayer*) –要加标记的玩家
- **mark** (str) –标记名称
- **count?** (integer) –要增加的数量，默认为 1

removePlayerMark(*player*, *mark*, *count?*)

将一名玩家的 mark 标记减少 count 个。

Parameters

- **player** (*ServerPlayer*) –要减标记的玩家
- **mark** (str) –标记名称
- **count?** (integer) –要减少的数量，默认为 1

setCardMark(*card*, *mark*, *value*)

将一张卡牌的某种标记数量相应的值。

在设置之后，会通知所有客户端也更新一下标记的值。之后的两个相同

Parameters

- **card** (*Card*) –要被更新标记的那张牌
- **mark** (str) –标记的名称
- **value** (any) –要设为的值，其实也可以设为字符串

addCardMark(*card*, *mark*, *count?*)

将一张卡牌的 mark 标记增加 count 个。

Parameters

- **card** (*Card*) –要被增加标记的那张牌
- **mark** (str) –标记名称
- **count?** (integer) –要增加的数量，默认为 1

removeCardMark(*card*, *mark*, *count?*)

将一名玩家的 mark 标记减少 count 个。

Parameters

- **card** (*Card*) –要被减少标记的那张牌
- **mark** (str) –标记名称
- **count?** (integer) –要减少的数量，默认为 1

setPlayerProperty(*player*)

Parameters

player (*ServerPlayer*) –

setTag(*tag_name*, *value*)

将房间中某个 tag 设为特定值。

当在编程中想在服务端搞点全局变量的时候哦，不要自己设置全局变量或者上值，而是应该使用 room 的 tag。

Parameters

- **tag_name** (str) –tag 名字

- **value** (any) –值

getTag(*tag_name*)

获得某个 tag 的值。

Parameters

tag_name (str) –tag 名字

removeTag(*tag_name*)

删除某个 tag。

Parameters

tag_name (str) –tag 名字

setBanner(*name*, *value*)

Parameters

- **name** (any) –
- **value** (any) –

setPlayerGeneral(*player*, *general*, *changeKingdom?*, *noBroadcast?*)

Parameters

- **player** (*ServerPlayer*) –
- **general** (str) –
- **changeKingdom?** (boolean) –
- **noBroadcast?** (boolean) –

setDeputyGeneral(*player*, *general*)

Parameters

- **player** (*ServerPlayer*) –
- **general** (str) –

changeHero(*player*, *new_general*, *full?*, *isDeputy?*, *sendLog?*, *maxHpChange?*)

Parameters

- **player** (*ServerPlayer*) –要换将的玩家
- **new_general** (str) –要变更的武将，若不存在则变身为孙策，孙策不存在变身为士兵
- **full?** (boolean) –是否血量满状态变身
- **isDeputy?** (boolean) –是否变的是副将
- **sendLog?** (boolean) –是否发 Log
- **maxHpChange?** (boolean) –是否改变体力上限，默认改变

changeKingdom(*player*, *kingdom*, *sendLog?*)

Parameters

- **player** (*ServerPlayer*) –要变更势力的玩家
- **kingdom** (str) –要变更的势力
- **sendLog?** (boolean) –是否发 Log

broadcastProperty(*player*, *property*)

向所有角色广播一名角色的某个 property，让大家都知道

Parameters

- **player** (*ServerPlayer*) –要被广而告之的那名角色
- **property** (str) –这名角色的某种属性，像是”hp”之类的，其实就是 Player 类的属性名

notifyProperty(*p*, *player*, *property*)

将 player 的属性 property 告诉 p。

Parameters

- **p** (*ServerPlayer*) –要被告知相应属性的那名玩家
- **player** (*ServerPlayer*) –拥有那个属性的玩家
- **property** (str) –属性名称

doBroadcastNotify(*command*, *jsonData*, *players?*)

向多名玩家广播一条消息。

Parameters

- **command** (str) –发出这条消息的消息类型
- **jsonData** (str) –消息的数据，一般是 JSON 字符串，也可以是普通字符串，取决于 client 怎么处理了
- **players?** (list[ServerPlayer]) –要告知的玩家列表，默认为所有人

doRequest(*player*, *command*, *jsonData*, *wait?*)

向某个玩家发起一次 Request。

Parameters

- **player** (*ServerPlayer*) –发出这个请求的目标玩家
- **command** (str) –请求的类型
- **jsonData** (str) –请求的数据
- **wait?** (boolean) –是否要等待答复，默认为 true

Returns

收到的答复，如果 wait 为 false 的话就返回 nil

Return type
str

doBroadcastRequest(*command*, *players?*, *jsonData?*)

向多名玩家发出请求。

Parameters

- **command** (str) –请求类型
- **players?** (list[ServerPlayer]) –发出请求的玩家列表
- **jsonData?** (str) –请求数据

doRaceRequest(*command*, *players*, *jsonData*)

向多名玩家发出竞争请求。

他们都可以做出答复，但是服务器只认可第一个做出回答的角色。

返回获胜的角色，可以通过属性获得回复的具体内容。

Parameters

- **command** (str) –请求类型
- **players** (list[ServerPlayer]) –要竞争这次请求的玩家列表
- **jsonData** (str) –请求数据

Returns
? @ 在这次竞争请求中获胜的角色，可能是 nil

Return type
ServerPlayer

delay(*ms*)

延迟一段时间。

这个函数不应该在请求处理协程中使用。

Parameters
ms (integer) –要延迟的毫秒数

notifyMoveCards(*players?*, *card_moves*, *forceVisible?*)

向多名玩家告知一次移牌行为。

Parameters

- **players?** (list[ServerPlayer]) –要被告知的玩家列表，默认为全员
- **card_moves** (list[CardsMoveStruct]) –要告知的移牌信息列表
- **forceVisible?** (boolean) –是否让所有牌对告知目标可见

notifyMoveFocus(*players*, *command*)

将焦点转移给一名或者多名角色，并广而告之。

形象点说，就是在那些玩家下面显示一个“弃牌思考中…”之类的烧条提示。

Parameters

- **players** (ServerPlayer or list[ServerPlayer]) –要获得焦点的一名或者多名角色
- **command** (str) –烧条的提示文字

sendLog(*log*)

向战报中发送一条 log。

Parameters

log (*LogMessage*) –Log 的实际内容

sendFootnote(*ids*, *log*)

Parameters

- **ids** (any) –
- **log** (any) –

sendCardVirtName(*ids*, *name*)

Parameters

- **ids** (any) –
- **name** (any) –

doAnimate(*type*, *data*, *players?*)

播放某种动画效果给 players 看。

Parameters

- **type** (str) –动画名字
- **data** (any) –这个动画附加的额外信息，在这个函数将会被转成 json 字符串
- **players?** (list[ServerPlayer]) –要观看动画的玩家们，默认为全员

setEmotion(*player*, *name*)

在 player 脸上展示名为 name 的 emotion 动效。

这就是“杀”、“闪”之类的那个动画。

Parameters

- **player** (*ServerPlayer*) –被播放动画的那个角色
- **name** (str) –emotion 名字，可以是一个路径

setCardEmotion(*cid*, *name*)

在一张 card 上播放一段 emotion 动效。

这张 card 必须在处理区里面，或者至少客户端觉得它在处理区。

Parameters

- **cid** (integer) –被播放动效的那个牌的 id
- **name** (str) –emotion 名字，可以是一个路径

doSuperLightBox(*path*, *extra_data*)

播放一个全屏大动画。可以自己指定 qml 文件路径和额外的信息。

Parameters

- **path** (str) –qml 文件的路径，有默认值
- **extra_data** (any) –要传递的额外信息

sendLogEvent(*type*, *data*, *players*)

基本上是个不常用函数就是了

Parameters

- **type** (any) –
- **data** (any) –
- **players** (any) –

broadcastSkillInvoke(*skill_name*, *index?*)

播放技能的语音。

Parameters

- **skill_name** (nil) –技能名
- **index?** (integer) –语音编号，默认为-1（也就是随机播放）

broadcastPlaySound(*path*)

播放一段音频。

Parameters

path (str) –音频文件路径

notifySkillInvoked(*player*, *skill_name*, *skill_type?*)

在 player 的脸上播放技能发动的特效。

与此同时，在战报里面发一条“xxx 发动了 xxx”

Parameters

- **player** (*ServerPlayer*) –发动技能的那个玩家
- **skill_name** (str) –技能名
- **skill_type?** (str) –技能的动画效果，默认是那个技能的 anim_type

doIndicate(*source*, *targets*)

播放从 source 指到 targets 的指示线效果。

Parameters

- **source** (integer) –指示线开始的那个玩家的 id
- **targets** (list[integer]) –指示线目标玩家的 id 列表

askForUseActiveSkill(*player*, *skill_name*, *prompt?*, *cancelable?*, *extra_data?*, *no_indicate?*)

询问 player 是否要发动一个主动技。

如果发动的话，那么会执行一下技能的 onUse 函数，然后返回选择的牌和目标等。

Parameters

- **player** (*ServerPlayer*) –询问目标
- **skill_name** (str) –主动技的技能名
- **prompt?** (str) –烧条上面显示的提示文本内容
- **cancelable?** (boolean) –是否可以点取消
- **extra_data?** (table) –额外信息，因技能而异了
- **no_indicate?** (boolean) –是否不显示指示线

Returns

?

Return type

boolean or table

askForDiscard(*player*, *minNum*, *maxNum*, *includeEquip?*, *skillName?*, *cancelable?*, *pattern?*, *prompt?*, *skipDiscard?*, *no_indicate?*)

询问一名角色弃牌。

在这个函数里面牌已经被弃掉了（除非 skipDiscard 为 true）。

Parameters

- **player** (*ServerPlayer*) –弃牌角色
- **minNum** (integer) –最小值
- **maxNum** (integer) –最大值
- **includeEquip?** (boolean) –能不能弃装备区?
- **skillName?** (str) –引发弃牌的技能名
- **cancelable?** (boolean) –能不能点取消?
- **pattern?** (str) –弃牌需要符合的规则
- **prompt?** (str) –提示信息

- **skipDiscard?** (boolean) –是否跳过弃牌（即只询问选择可以弃置的牌）
- **no_indicate?** (boolean) –是否不显示指示线

Returns
弃掉的牌的 id 列表，可能是空的

Return type
list[integer]

askForChoosePlayers(*player*, *targets*, *minNum*, *maxNum*, *prompt?*, *skillName?*, *cancelable?*, *no_indicate?*)

询问一名玩家从 targets 中选择若干名玩家出来。

Parameters

- **player** (*ServerPlayer*) –要做选择的玩家
- **targets** (list[integer]) –可以选的目标范围，是玩家 id 数组
- **minNum** (integer) –最小值
- **maxNum** (integer) –最大值
- **prompt?** (str) –提示信息
- **skillName?** (str) –技能名
- **cancelable?** (boolean) –能否点取消
- **no_indicate?** (boolean) –是否不显示指示线

Returns
选择的玩家 id 列表，可能为空

Return type
list[integer]

askForCard(*player*, *minNum*, *maxNum*, *includeEquip?*, *skillName?*, *cancelable?*, *pattern?*, *prompt?*, *expand_pile?*, *no_indicate?*)

询问一名玩家选择自己的几张牌。

与 askForDiscard 类似，但是不对选择的牌进行操作就是了。

Parameters

- **player** (*ServerPlayer*) –要询问的玩家
- **minNum** (integer) –最小值
- **maxNum** (integer) –最大值
- **includeEquip?** (boolean) –能不能选装备
- **skillName?** (str) –技能名
- **cancelable?** (boolean) –能否点取消

- **pattern?** (str) –选牌规则
- **prompt?** (str) –提示信息
- **expand_pile?** (str) –可选私人牌堆名称
- **no_indicate?** (boolean) –是否不显示指示线

Returns
选择的牌的 id 列表，可能是空的

Return type
list[integer]

askForChooseCardAndPlayers(*player*, *targets*, *minNum*, *maxNum*, *pattern?*, *prompt?*, *cancelable?*, *no_indicate?*)

询问玩家选择 1 张牌和若干名角色。

返回两个值，第一个是选择的目标列表，第二个是选择的那张牌的 id

Parameters

- **player** (*ServerPlayer*) –要询问的玩家
- **targets** (list[integer]) –选择目标的 id 范围
- **minNum** (integer) –选目标最小值
- **maxNum** (integer) –选目标最大值
- **pattern?** (str) –选牌规则
- **prompt?** (str) –提示信息
- **cancelable?** (boolean) –能否点取消
- **no_indicate?** (boolean) –是否不显示指示线

Return type
list[integer] or integer

askForChooseCardsAndPlayers(*player*, *minCardNum*, *maxCardNum*, *targets*, *minTargetNum*, *maxTargetNum*, *pattern?*, *prompt?*, *cancelable?*, *no_indicate?*)

询问玩家选择 X 张牌和 Y 名角色。

返回两个值，第一个是选择的目标列表，第二个是选择的牌 id 列表

Parameters

- **player** (*ServerPlayer*) –要询问的玩家
- **minCardNum** (integer) –选卡牌最小值
- **maxCardNum** (integer) –选卡牌最大值
- **targets** (list[integer]) –选择目标的 id 范围
- **minTargetNum** (integer) –选目标最小值

- **maxTargetNum** (integer) –选目标最大值
- **pattern?** (str) –选牌规则
- **prompt?** (str) –提示信息
- **cancelable?** (boolean) –能否点取消
- **no_indicate?** (boolean) –是否不显示指示线

Return type
list[integer] or list[integer]

askForYiji(*player*, *cards?*, *targets?*, *skillName?*, *minNum?*, *maxNum?*, *prompt?*, *expand_pile?*, *skipMove?*, *single_max?*)

询问将卡牌分配给任意角色。

Parameters

- **player** (*ServerPlayer*) –要询问的玩家
- **cards?** (list[integer]) –要分配的卡牌。默认拥有的所有牌
- **targets?** (list[ServerPlayer]) –可以获得卡牌的角色。默认所有存活角色
- **skillName?** (str) –技能名，影响焦点信息。默认为“分配”
- **minNum?** (integer) –最少交出的卡牌数，默认 0
- **maxNum?** (integer) –最多交出的卡牌数，默认所有牌
- **prompt?** (str) –询问提示信息
- **expand_pile?** (str) –可选私人牌堆名称，如要分配你武将牌上的牌请填写
- **skipMove?** (boolean) –是否跳过移动。默认不跳过
- **single_max?** (integer or table) –限制每人能获得的最大牌数。输入整数或 (以角色 id 为键以整数为值) 的表

Returns
<integer[]> @ 返回一个表，键为角色 id，值为分配给其的牌 id 数组

Return type
table

getNGenerals(*n*, *position?*)

抽个武将

同 getNCards，抽出来就没有了，所以记得放回去。

Parameters

- **n** (number) –数量
- **position?** (str) –位置，top/bottom，默认 top

Returns
武将名数组

Return type
`list[str]`

`returnToGeneralPile`(*g*, *position?*)

把武将牌塞回去（……）

Parameters

- **g** (`list[str]`) –武将名数组
- **position?** (`str`) –位置，top/bottom，默认 bottom

Returns
是否成功

Return type
`boolean`

`findGeneral`(*name*)

抽特定名字的武将（抽了就没了）

Parameters
name (`str`) –武将 name，如找不到则查找 truename，再找不到则返回 nil

Returns
? @ 抽出的武将名

Return type
`str`

`findGenerals`(*func*, *n?*)

自上而下抽符合特定情况的 N 个武将（抽了就没了）

Parameters

- **func** (`fun(str):any`) –武将筛选函数
- **n?** (`integer`) –抽取数量，数量不足则直接抽干净

Returns
武将组合，可能为空

Return type
`list[str]`

`askForGeneral`(*player*, *generals*, *n*, *noConvert?*)

询问玩家选择一名武将。

Parameters

- **player** (*ServerPlayer*) –询问目标
- **generals** (`list[str]`) –可选武将

- **n** (integer) –可选数量，默认为 1
- **noConvert?** (boolean) –可否变更，默认可

Returns
选择的武将

Return type
str or list[str]

askForChooseKingdom(*players?*)

询问玩家若为神将、双势力需选择一个势力。

Parameters
players? (list[ServerPlayer]) –询问目标

askForCardChosen(*chooser*, *target*, *flag*, *reason*, *prompt?*)

询问 chooser，选择 target 的一张牌。

Parameters

- **chooser** (*ServerPlayer*) –要被询问的人
- **target** (*ServerPlayer*) –被选牌的人
- **flag** (any) –用” hej” 三个字母的组合表示能选择哪些区域, h 手牌区, e - 装备区, j - 判定区
- **reason** (str) –原因，一般是技能名
- **prompt?** (str) –提示信息

Returns
选择的卡牌 id

Return type
integer

askForCardsChosen(*chooser*, *target*, *min*, *max*, *flag*, *reason*, *prompt?*)

完全类似 askForCardChosen，但是可以选择多张牌。

相应的，返回的是 id 的数组而不是单个 id。

Parameters

- **chooser** (*ServerPlayer*) –要被询问的人
- **target** (*ServerPlayer*) –被选牌的人
- **min** (integer) –最小选牌数
- **max** (integer) –最大选牌数
- **flag** (any) –用” hej” 三个字母的组合表示能选择哪些区域, h 手牌区, e - 装备区, j - 判定区
- **reason** (str) –原因，一般是技能名
- **prompt?** (str) –提示信息

Returns
选择的 id

Return type
`list[integer]`

askForPoxi(*player*, *poxi_type*, *data*, *extra_data*, *cancelable?*)

谋 askForCardsChosen，需使用 Fk:addPoxiMethod 定义好方法

选卡规则和返回值啥的全部自己想办法解决，data 填入所有卡的列表（类似 ui.card_data）

注意一定要返回一个表，毕竟本质上是选卡函数

Parameters

- **player** (*ServerPlayer*) –
- **poxi_type** (`str`) –
- **data** (`any`) –
- **extra_data** (`any`) –
- **cancelable?** (`boolean`) –

Return type
`list[integer]`

askForChoice(*player*, *choices*, *skill_name?*, *prompt?*, *detailed?*, *all_choices?*)

询问一名玩家从众多选项中选择一个。

Parameters

- **player** (*ServerPlayer*) –要询问的玩家
- **choices** (`list[str]`) –可选选项列表
- **skill_name?** (`str`) –技能名
- **prompt?** (`str`) –提示信息
- **detailed?** (`boolean`) –选项详细描述
- **all_choices?** (`list[str]`) –所有选项（不可选变灰）

Returns
选择的选项

Return type
`str`

askForChoices(*player*, *choices*, *minNum*, *maxNum*, *skill_name?*, *prompt?*, *cancelable?*, *detailed?*, *all_choices?*)

询问一名玩家从众多选项中勾选任意项。

Parameters

- **player** (*ServerPlayer*) –要询问的玩家

- **choices** (list[str]) –可选选项列表
- **minNum** (number) –最少选择项数
- **maxNum** (number) –最多选择项数
- **skill_name?** (str) –技能名
- **prompt?** (str) –提示信息
- **cancelable?** (boolean) –是否可取消
- **detailed?** (boolean) –选项详细描述
- **all_choices?** (list[str]) –所有选项（不可选变灰）

Returns
选择的选项

Return type
list[str]

askForSkillInvoke(*player*, *skill_name*, *data?*, *prompt?*)

询问玩家是否发动技能。

Parameters

- **player** (*ServerPlayer*) –要询问的玩家
- **skill_name** (str) –技能名
- **data?** (any) –未使用
- **prompt?** (str) –提示信息

Return type
boolean

getUseExtraTargets(*data*, *bypass_distances*, *use_AimGroup*)

Parameters

- **data** (*CardUseStruct*) –使用事件的 data
- **bypass_distances** (boolean) –? @ 是否无距离关系的限制
- **use_AimGroup** (boolean) –? @ 某些场合需要使用 AimGroup，by smart Ho-spair

Returns
返回满足条件的 player 的 id 列表

Return type
list[integer]

askForAddTarget(*player*, *targets*, *num*, *can_minus*, *distance_limited*, *prompt*, *skillName*, *data*)

Parameters

- **player** (*ServerPlayer*) –执行的玩家
- **targets** (list[ServerPlayer]) –可选的目标范围
- **num** (integer) –可选的目标数
- **can_minus** (boolean) –是否可减少
- **distance_limited** (boolean) –是否受距离限制
- **prompt** (str) –提示信息
- **skillName** (str) –技能名
- **data** (*CardUseStruct*) –使用数据

askForGuanxing(*player*, *cards*, *top_limit?*, *bottom_limit?*, *customNotify?*, *noPut?*, *areaNames?*)

询问玩家对若干牌进行观星。

观星完成后，相关的牌会被置于牌堆顶或者牌堆底。所以这些 cards 最好不要来自牌堆，一般先用 getNCards 从牌堆拿出一些牌。param prompt? string @ 观星框的标题 (暂时雪藏)

Parameters

- **player** (*ServerPlayer*) –要询问的玩家
- **cards** (list[integer]) –可以被观星的卡牌 id 列表
- **top_limit?** (list[integer]) –置于牌堆顶的牌的限制 (下限, 上限)，不填写则不限
- **bottom_limit?** (list[integer]) –置于牌堆底的牌的限制 (下限, 上限)，不填写则不限
- **customNotify?** (str) –自定义读条操作提示
- **noPut?** (boolean) –是否进行放置牌操作
- **areaNames?** (list[str]) –左侧提示信息

Returns

<" top" |" bottom" , integer[]>

Return type

table

askForExchange(*player*, *piles*, *piles_name*, *customNotify?*)

询问玩家任意交换几堆牌堆。

Parameters

- **player** (*ServerPlayer*) –要询问的玩家
- **piles** (dict[str, list[integer]]) –卡牌 id 列表的列表，也就是……几堆牌堆的集合
- **piles_name** (list[str]) –牌堆名，必须一一对应，否则统一替换为“牌堆 X”

- **customNotify?** (str) –自定义读条操作提示

Return type
dict[str, list[integer]]

handleUseCardReply(*player*, *data*)

平时写 DIY 用不到的函数。

Parameters

- **player** (*ServerPlayer*) –
- **data** (str) –

Return type
CardUseStruct

askForUseCard(*player*, *card_name?*, *pattern?*, *prompt?*, *cancelable?*, *extra_data?*, *event_data?*)

询问玩家使用一张牌。

Parameters

- **player** (*ServerPlayer*) –要询问的玩家
- **card_name?** (str) –使用牌的牌名，若 pattern 指定了则可随意写，它影响的是烧条的提示信息
- **pattern?** (str) –使用牌的规则，默认就是 card_name 的值
- **prompt?** (str) –提示信息
- **cancelable?** (boolean) –能否点取消
- **extra_data?** (*UseExtraData*) –额外信息
- **event_data?** (*CardEffectEvent*) –事件信息

Returns
? @ 返回关于本次使用牌的数据，以便后续处理

Return type
CardUseStruct

askForResponse(*player*, *card_name*, *pattern?*, *prompt?*, *cancelable?*, *extra_data?*, *effectData?*)

询问一名玩家打出一张牌。

Parameters

- **player** (*ServerPlayer*) –要询问的玩家
- **card_name** (str) –牌名
- **pattern?** (str) –牌的规则
- **prompt?** (str) –提示信息
- **cancelable?** (boolean) –能否取消

- **extra_data?** (any) –额外数据
- **effectData?** (*CardEffectEvent*) –关联的卡牌生效流程

Returns
? @ 打出的牌

Return type
Card

askForNullification(*players*, *card_name*, *pattern*, *prompt?*, *cancelable?*, *extra_data?*)

同时询问多名玩家是否使用某一张牌。

函数名字虽然是“询问无懈可击”，不过其实也可以给别的牌用就是了。

Parameters

- **players** (list[ServerPlayer]) –要询问的玩家列表
- **card_name** (str) –询问的牌名，默认为无懈
- **pattern** (str) –牌的规则
- **prompt?** (str) –提示信息
- **cancelable?** (boolean) –能否点取消
- **extra_data?** (any) –额外信息

Returns
? @ 最终决胜出的卡牌使用信息

Return type
CardUseStruct

askForAG(*player*, *id_list*, *cancelable?*, *reason?*)

询问玩家从 AG 中选择一张牌。

Parameters

- **player** (*ServerPlayer*) –要询问的玩家
- **id_list** (list[integer] or list[Card]) –可选的卡牌列表
- **cancelable?** (boolean) –能否点取消
- **reason?** (str) –原因

Returns
选择的卡牌

Return type
integer

fillAG(*player*, *id_list*, *disable_ids?*)

给 player 发一条消息，在他的窗口中用一系列卡牌填充一个 AG。

Parameters

- **player** (*ServerPlayer*) –要通知的玩家
- **id_list** (`list[integer] or list[Card]`) –要填充的卡牌
- **disable_ids?** (`list[integer] or list[Card]`) –未使用

takeAG(*taker*, *id*, *notify_list?*)

告诉一些玩家，AG 中的牌被 taker 取走了。

Parameters

- **taker** (*ServerPlayer*) –拿走牌的玩家
- **id** (`integer`) –被拿走的牌
- **notify_list?** (`list[ServerPlayer]`) –要告知的玩家，默认为全员

closeAG(*player?*)

关闭 player 那侧显示的 AG。

若不传参（即 player 为 nil），那么关闭所有玩家的 AG。

Parameters

player? (*ServerPlayer*) –要关闭 AG 的玩家

askForMiniGame(*players*, *focus*, *game_type*, *data_table*)

Parameters

- **players** (`list[ServerPlayer]`) –
- **focus** (`str`) –
- **game_type** (`str`) –
- **data_table** (`dict[integer, any]`) –对应每个 player

askForCustomDialog(*player*, *focustxt*, *qmlPath*, *extra_data*)

Parameters

- **player** (*ServerPlayer*) –
- **focustxt** (`str`) –
- **qmlPath** (`str`) –
- **extra_data** (`any`) –

Return type

`str`

askForMoveCardInBoard(*player*, *targetOne*, *targetTwo*, *skillName*, *flag?*, *moveFrom?*, *excludeIds?*)

Parameters

- **player** (*ServerPlayer*) –移动的操作
- **targetOne** (*ServerPlayer*) –移动的目标 1 玩家

- **targetTwo** (*ServerPlayer*) –移动的目标 2 玩家
- **skillName** (str) –技能名
- **flag?** (str) –限定可移动的区域，值为 nil（装备区和判定区）、‘e’ 或 ‘j’
- **moveFrom?** (*ServerPlayer*) –是否只是目标 1 移动给目标 2
- **excludeIds?** (list[integer]) –本次不可移动的卡牌 id

Returns
<” card” |” from” |” to” >? @ 选择的卡牌、起点玩家 id 和终点玩家 id 列表

Return type
table

askForChooseToMoveCardInBoard(*player*, *prompt*, *skillName*, *cancelable?*, *flag?*, *no_indicate?*)

询问一名玩家从 targets 中选择出若干名玩家来移动场上的牌。

Parameters

- **player** (*ServerPlayer*) –要做选择的玩家
- **prompt** (str) –提示信息
- **skillName** (str) –技能名
- **cancelable?** (boolean) –是否可以取消选择
- **flag?** (str) –限定可移动的区域，值为 nil（装备区和判定区）、‘e’ 或 ‘j’
- **no_indicate?** (boolean) –是否不显示指示线

Returns
选择的玩家 id 列表，可能为空

Return type
list[integer]

useCard(*cardUseEvent*)

根据卡牌使用数据，去实际使用这个卡牌。

Parameters
cardUseEvent (*CardUseStruct*) –使用数据

Return type
boolean

doCardUseEffect(*cardUseEvent*)

对卡牌使用数据进行生效

Parameters
cardUseEvent (*CardUseStruct*) –

doCardEffect(*cardEffectEvent*)

对卡牌效果数据进行生效

Parameters

cardEffectEvent (*CardEffectEvent*) –

handleCardEffect(*cardEffectEvent*)

Parameters

cardEffectEvent (*CardEffectEvent*) –

responseCard(*cardResponseEvent*)

对“打出牌”进行处理

Parameters

cardResponseEvent (*CardResponseEvent*) –

useVirtualCard(*card_name*, *subcards?*, *from*, *tos*, *skillName?*, *extra?*)

Parameters

- **card_name** (str) –想要视为使用的牌名
- **subcards?** (list[integer]) –子卡，可以留空或者直接 nil
- **from** (*ServerPlayer*) –使用来源
- **tos** (ServerPlayer or list[ServerPlayer]) –目标角色（列表）
- **skillName?** (str) –技能名
- **extra?** (boolean) –是否不计入次数

moveCards(...)

传入一系列移牌信息，去实际移动这些牌

Parameters

vararg (CardsMoveInfo) –

Returns

?

Return type

boolean

obtainCard(*player*, *cid*, *unhide?*, *reason?*, *proposer?*)

让一名玩家获得一张牌

Parameters

- **player** (integer or ServerPlayer) –要拿牌的玩家
- **cid** (integer or Card or list[integer]) –要拿到的卡牌
- **unhide?** (boolean) –是否明着拿
- **reason?** (CardMoveReason) –卡牌移动的原因

- **proposer?** (integer) –移动操作者的 id

drawCards(*player*, *num*, *skillName?*, *fromPlace?*)

让玩家摸牌

Parameters

- **player** (*ServerPlayer*) –摸牌的玩家
- **num** (integer) –摸牌数
- **skillName?** (str) –技能名
- **fromPlace?** (str) –摸牌的位置，”top”或者“bottom”

Returns

摸到的牌

Return type

list[integer]

moveCardTo(*card*, *to_place*, *target?*, *reason?*, *skill_name?*, *special_name?*, *visible?*, *proposer?*)

将一张或多张牌移动到某处

Parameters

- **card** (integer or list[integer] or Card or list[Card]) –要移动的牌
- **to_place** (integer) –移动的目标位置
- **target?** (*ServerPlayer*) –移动的目标角色
- **reason?** (integer) –移动时使用的移牌原因
- **skill_name?** (str) –技能名
- **special_name?** (str) –私人牌堆名
- **visible?** (boolean) –是否明置
- **proposer?** (integer) –移动操作者的 id

doYiji(*room*, *list*, *proposer?*, *skillName?*)

将一些卡牌同时分配给一些角色。

Parameters

- **room** (*Room*) –房间
- **list** (table) –<integer[]> @ 分配牌和角色的数据表，键为角色 id，值为分配给其的牌 id 数组
- **proposer?** (integer) –操作者的 id。默认为空
- **skillName?** (str) –技能名。默认为“分配”

Returns

<integer[]> @ 返回成功分配的卡牌

Return type
table

moveCardIntoEquip(*target*, *cards*, *skillName?*, *convert?*, *proposer?*)

将一张牌移动至某角色的装备区，若不合法则置入弃牌堆。目前没做相同副类别装备同时置入的适配 (甘露神典韦)

Parameters

- **target** (*ServerPlayer*) –接受牌的角色
- **cards** (integer or list[integer]) –移动的牌
- **skillName?** (str) –技能名
- **convert?** (boolean) –是否可以替换装备（默认可以）
- **proposer?** (*ServerPlayer*) –操作者

changeHp(*player*, *num*, *reason?*, *skillName?*, *damageStruct?*)

改变一名玩家的体力。

Parameters

- **player** (*ServerPlayer*) –玩家
- **num** (integer) –变化量
- **reason?** (str) –原因
- **skillName?** (str) –技能名
- **damageStruct?** (*DamageStruct*) –伤害数据

Return type
boolean

changeShield(*player*, *num*)

改变玩家的护甲数

Parameters

- **player** (*ServerPlayer*) –
- **num** (integer) –变化量

loseHp(*player*, *num*, *skillName?*)

令一名玩家失去体力。

Parameters

- **player** (*ServerPlayer*) –玩家
- **num** (integer) –失去的数量
- **skillName?** (str) –技能名

Return type
boolean

changeMaxHp(*player*, *num*)

改变一名玩家的体力上限。

Parameters

- **player** (*ServerPlayer*) –玩家
- **num** (integer) –变化量

Return type

boolean

damage(*damageStruct*)

根据伤害数据造成伤害。

Parameters

damageStruct (*DamageStruct*) –

Return type

boolean

recover(*recoverStruct*)

根据回复数据回复体力。

Parameters

recoverStruct (*RecoverStruct*) –

Return type

boolean

enterDying(*dyingStruct*)

根据濒死数据让人进入濒死。

Parameters

dyingStruct (*DyingStruct*) –

killPlayer(*deathStruct*)

根据死亡数据杀死角色。

Parameters

deathStruct (*DeathStruct*) –

handleAddLoseSkills(*player*, *skill_names*, *source_skill?*, *no_trigger?*)

令一名玩家获得/失去技能。

skill_names 是字符串数组或者用管道符号 (|) 分割的字符串。

每个 skill_name 都是要获得的技能的名。如果在 skill_name 前面加上” -”，那就是失去技能。

Parameters

- **player** (*ServerPlayer*) –玩家
- **skill_names** (list[str] or str) –要获得/失去的技能
- **source_skill?** (str or Skill) –源技能

- **no_trigger?** (boolean) –是否不触发相关时机

judge(*data*)

根据判定数据进行判定。判定的结果直接保存在这个数据中。

Parameters

data (*JudgeStruct*) –

retrial(*card*, *player*, *judge*, *skillName?*, *exchange?*)

改判。

Parameters

- **card** (*Card*) –改判的牌
- **player** (*ServerPlayer*) –改判的玩家
- **judge** (*JudgeStruct*) –要被改判的判定数据
- **skillName?** (str) –技能名
- **exchange?** (boolean) –是否要替换原有判定牌（即类似鬼道那样）

throwCard(*card_ids*, *skillName?*, *who*, *thrower?*)

弃置一名角色的牌。

Parameters

- **card_ids** (list[integer] or integer) –被弃掉的牌
- **skillName?** (str) –技能名
- **who** (*ServerPlayer*) –被弃牌的人
- **thrower?** (*ServerPlayer*) –弃别人牌的人

recastCard(*card_ids*, *who*, *skillName?*)

重铸一名角色的牌。

Parameters

- **card_ids** (list[integer]) –被重铸的牌
- **who** (*ServerPlayer*) –重铸的角色
- **skillName?** (str) –技能名，默认为“重铸”

Returns

摸到的牌

Return type

list[integer]

pindian(*pindianData*)

根据拼点信息开始拼点。

Parameters

pindianData (*PindianStruct*) –

adjustSeats()

swapSeat(*a*, *b*)

Parameters

- **a** (*ServerPlayer*) –
- **b** (*ServerPlayer*) –

shuffleDrawPile()

洗牌。

useSkill(*player*, *skill*, *effect_cb*)

使用技能。先增加技能发动次数，再执行相应的函数。

Parameters

- **player** (*ServerPlayer*) –发动技能的玩家
- **skill** (*Skill*) –发动的技能
- **effect_cb** (fun()) –实际要调用的函数

revivePlayer(*player*, *sendLog?*)

Parameters

- **player** (*ServerPlayer*) –
- **sendLog?** (boolean) –

gameOver(*winner*)

结束一局游戏。

Parameters

winner (str) –获胜的身份，空字符串表示平局

getSubcardsByRule(*card*, *fromAreas?*)

Parameters

- **card** (*Card*) –
- **fromAreas?** (list[CardArea]) –

Return type

list[integer]

getCardsFromPileByRule(*pattern*, *num?*, *fromPile?*)

Parameters

- **pattern** (str) –
- **num?** (number) –
- **fromPile?** (str) – 查找的来源区域，值为 drawPile|discardPile|allPiles

Returns
id 列表可能空

Return type
`list[integer]`

`canMoveCardInBoard`(*flag?*, *players?*, *excludeIds?*)

Parameters

- **flag?** (`str`) –
- **players?** (`list[ServerPlayer]`) –
- **excludeIds?** (`list[integer]`) –

Returns
玩家 id 列表可能为空

Return type
`list[integer]`

`printCard`(*name*, *suit?*, *number?*)

现场印卡。当然了，这个卡只和这个房间有关。

Parameters

- **name** (`str`) –牌名
- **suit?** (`Suit`) –花色
- **number?** (`integer`) –点数

Return type
Card

`updateQuestSkillState`(*player*, *skillName*, *failed?*)

刷新使命技状态

Parameters

- **player** (*ServerPlayer*) –
- **skillName** (`Suit`) –
- **failed?** (`boolean`) –

`abortPlayerArea`(*player*, *playerSlots*)

废除区域

Parameters

- **player** (*ServerPlayer*) –
- **playerSlots** (`str or list[str]`) –

`resumePlayerArea`(*player*, *playerSlots*)

恢复区域

Parameters

- **player** (*ServerPlayer*) –
- **playerSlots** (str or list[str]) –

setPlayerRest(*player*, *roundNum*)

设置休整

Parameters

- **player** (*ServerPlayer*) –
- **roundNum** (integer) –

CHAPTER 54

ServerPlayer

class ServerPlayer: *Player*

serverplayer: fk.ServerPlayer

room: Room

next: ServerPlayer

request_data: str

mini_game_data: any

client_reply: str

default_reply: str

reply_ready: boolean

reply_cancel: boolean

phases: list[Phase]

skipped_phases: list[Phase]

phase_state: list[table]

phase_index: integer

role_shown: boolean

prelighted_skills: list[Skill]

ai: AI

ai_data: any

initialize(*_self*)

Parameters
_self (any) –

doNotify(*command*, *jsonData*)

Parameters

- **command** (str) –
- **jsonData** (str) –

doRequest(*command*, *jsonData*, *timeout?*)

Send a request to client, and allow client to reply within *timeout* seconds.

timeout must not be negative. If nil, room.timeout is used.

Parameters

- **command** (str) –
- **jsonData** (str) –
- **timeout?** (integer) –

waitForReply(*timeout*)

Wait for at most *timeout* seconds for reply from client.

If *timeout* is negative or **nil**, the function will wait forever until get reply.

Parameters
timeout (integer) –seconds to wait

Returns
JSON data

Return type
str

marshal(*player*, *observe?*)

Parameters

- **player** (*ServerPlayer*) –
- **observe?** (boolean) –

reconnect()

isAlive()

turnOver()

showCards(*cards*)

Parameters

cards (any) –

changePhase(*from_phase*, *to_phase*)

Parameters

- **from_phase** (Phase) –
- **to_phase** (Phase) –

gainAnExtraPhase(*phase*, *delay?*)

Parameters

- **phase** (Phase) –
- **delay?** (boolean) –

play(*phase_table?*)

Parameters

phase_table? (list[Phase]) –

skip(*phase*)

Parameters

phase (Phase) –

endPlayPhase()

当进行到出牌阶段空闲点时，结束出牌阶段。

gainAnExtraTurn(*delay?*, *skillName?*)

获得一个额外回合

Parameters

- **delay?** (boolean) –
- **skillName?** (str) –

insideExtraTurn()

当前是否处于额外的回合。

Return type

boolean

getCurrentExtraTurnReason()

当前额外回合的技能原因。

Return type

str

drawCards(*num*, *skillName?*, *fromPlace?*)

角色摸牌。

Parameters

- **num** (integer) –摸牌数
- **skillName?** (str) –技能名
- **fromPlace?** (str) –摸牌的位置，”top”或者“bottom”

Returns
摸到的牌

Return type
list[integer]

addToPile(*pile_name*, *card*, *visible?*, *skillName?*)

Parameters

- **pile_name** (str) –
- **card** (integer or Card) –
- **visible?** (boolean) –
- **skillName?** (str) –

bury()

throwAllCards(*flag*)

Parameters
flag (any) –

throwAllMarks()

clearPiles()

addVirtualEquip(*card*)

Parameters
card (any) –

removeVirtualEquip(*cid*)

Parameters
cid (any) –

addCardUseHistory(*cardName*, *num*)

Parameters

- **cardName** (any) –
- **num** (any) –

setCardUseHistory(*cardName*, *num*, *scope*)

Parameters

- **cardName** (any) –

- **num** (any) –
- **scope** (any) –

addSkillUseHistory(*cardName*, *num*)

Parameters

- **cardName** (any) –
- **num** (any) –

setSkillUseHistory(*cardName*, *num*, *scope*)

Parameters

- **cardName** (any) –
- **num** (any) –
- **scope** (any) –

setChainState(*chained*)

Parameters
chained (boolean) –

reset()

pindian(*tos*, *skillName*, *initialCard?*)
进行拼点。

Parameters

- **tos** (list[ServerPlayer]) –
- **skillName** (str) –
- **initialCard?** (*Card*) –

Return type
PindianStruct

broadcastSkillInvoke(*skill_name*, *index?*)
播放技能的语音。

Parameters

- **skill_name** (str) –技能名
- **index?** (integer) –语音编号，默认为-1（也就是随机播放）

addFakeSkill(*skill*)

Parameters
skill (Skill or str) –

`loseFakeSkill`(*skill*)

Parameters
skill (Skill or str) –

`isFakeSkill`(*skill*)

Parameters
skill (Skill or str) –

`prelightSkill`(*skill*, *isPrelight?*)

Parameters

- **skill** (str or Skill) –
- **isPrelight?** (boolean) –

`revealGeneral`(*isDeputy?*, *no_trigger?*)

Parameters

- **isDeputy?** (boolean) –
- **no_trigger?** (boolean) –

`revealGenerals`()

`revealBySkillName`(*skill_name*)

Parameters
skill_name (any) –

`hideGeneral`(*isDeputy*)

Parameters
isDeputy (any) –

`control`(*p*)

Parameters
p (*ServerPlayer*) –

`addBuddy`(*other*)

Parameters
other (any) –

`removeBuddy`(*other*)

Parameters
other (any) –

CHAPTER 55

GameLogic

class GameLogic: Object

room: Room

skill_table: dict[Event, list[TriggerSkill]]

skill_priority_table: dict[Event, list[number]]

refresh_skill_table: dict[Event, list[TriggerSkill]]

skills: list[str]

game_event_stack: Stack

cleaner_stack: Stack

role_table: list[str]

[]

all_game_events: list[GameEvent]

event_recorder: dict[integer, GameEvent]

current_event_id: integer

initialize(*room*)

Parameters

room (any) –

run()

assignRoles()

`chooseGenerals()`

`buildPlayerCircle()`

`broadcastGeneral()`

`prepareDrawPile()`

`attachSkillToPlayers()`

`prepareForStart()`

`action()`

`addTriggerSkill(skill)`

Parameters

skill (*TriggerSkill*) –

`trigger(event, target?, data?)`

Parameters

- **event** (Event) –
- **target?** (*ServerPlayer*) –
- **data?** (any) –

`start()`

`pushEvent(event)`

Parameters

event (*GameEvent*) –

`resumeEvent(event)`

Parameters

event (*GameEvent*) –

Returns

?

Return type

boolean or GameEvent

`getCurrentCleaner()`

Return type

GameEvent

`clearEvent(event)`

Parameters

event (*GameEvent*) –

getCurrentEvent()

Return type
GameEvent

getMostRecentEvent(*eventType*)

Parameters
eventType (integer) –

getCurrentSkillName()
如果当前事件刚好是技能生效事件，就返回那个技能名，否则返回空串。

getEventsOfScope(*eventType*, *n*, *func*, *scope*)

Parameters

- **eventType** (integer) –要查找的事件类型
- **n** (integer) –最多找多少个
- **func** (fun(GameEvent):boolean) –过滤用的函数
- **scope** (integer) –查询历史范围，只能是当前阶段/回合/轮次

Returns
找到的符合条件的所有事件，最多 n 个但不保证有 n 个

Return type
list[GameEvent]

getEventsByRule(*eventType*, *func*, *n*, *end_id*)

Parameters

- **eventType** (integer) –要查找的事件类型
- **func** (fun(GameEvent):boolean) –过滤用的函数
- **n** (integer) –最多找多少个
- **end_id** (integer) –查询历史范围：从最后的事件开始逆序查找直到 id 为 end_id 的事件（不含）

Returns
找到的符合条件的所有事件，最多 n 个但不保证有 n 个

Return type
list[GameEvent]

getActualDamageEvents(*n*, *func*, *scope?*, *end_id?*)
获取实际的伤害事件

Parameters

- **n** (integer) –最多找多少个
- **func** (fun(GameEvent):boolean) –过滤用的函数

- **scope?** (integer) –查询历史范围，只能是当前阶段/回合/轮次
- **end_id?** (integer) –查询历史范围：从最后的事件开始逆序查找直到 id 为 end_id 的事件（不含）

Returns
找到的符合条件的所有事件，最多 n 个但不保证有 n 个

Return type
list[GameEvent]

dumpEventStack(*detailed*)

Parameters
detailed (any) –

dumpAllEvents(*from*, *to*)

Parameters

- **from** (any) –
- **to** (any) –

breakEvent(*ret*)

Parameters
ret (any) –

breakTurn()

CHAPTER 56

GameEvent

class GameEvent: Object

id: integer
事件的 id，随着时间推移自动增加并分配给新事件

end_id: integer
事件的对应结束 id，如果整个事件中未插入事件，那么 end_id 就是自己的 id

room: Room
room 实例

event: integer
该事件对应的 EventType

data: any
事件的附加数据，视类型而定

parent: GameEvent
事件的父事件（栈中的上一层事件）

prepare_func: fun(GameEvent)
事件即将开始时执行的函数

main_func: fun(GameEvent)
事件的主函数

clear_func: fun(GameEvent)
事件结束时执行的函数

extra_clear_funcs: fun(GameEvent)

[] @ 事件结束时执行的自定义函数列表

exit_func: fun(GameEvent)

事件结束后执行的函数

extra_exit_funcs: fun(GameEvent)

[] @ 事件结束后执行的自定义函数

exec_ret: boolean

? @ exec 函数的返回值，可能不存在

status: str

ready, running, exiting, dead

interrupted: boolean

事件是否是因为被中断而结束的，可能是防止事件或者被杀

killed: boolean

事件因为终止一切结算而被中断（所谓的“被杀”）

initialize(*event*, ...)

Parameters

- **event** (any) –
- **vararg** (any) –

translate(*id*)

Parameters

id (any) –

addCleaner(*f*)

Parameters

f (any) –

addExitFunc(*f*)

Parameters

f (any) –

prependExitFunc(*f*)

Parameters

f (any) –

findParent(*eventType*, *includeSelf*)

Parameters

- **eventType** (any) –
- **includeSelf** (any) –

searchEvents(*eventType*, *n*, *func*, *endEvent?*)

Parameters

- **eventType** (integer) –要查找的事件类型
- **n** (integer) –最多找多少个
- **func** (fun(GameEvent):boolean) –? @ 过滤用的函数
- **endEvent?** (*GameEvent*) –区间终止点，默认为本事件结束

Returns

找到的符合条件的所有事件，最多 n 个但不保证有 n 个

Return type

list[GameEvent]

exec()

shutdown()

CHAPTER 57

Trigger Data

这里记录各种 data 的数据，注意它们在源码之中仅仅只是一些注释而已。

57.1 CardsMoveStruct

class CardsMoveStruct

moveInfo: list[MoveInfo]

from: None

to: None

toArea: CardArea

moveReason: CardMoveReason

proposer: None

skillName: None

moveVisible: None

specialName: None

specialVisible: None

drawPilePosition: None

移至牌堆的索引位置，值为-1 代表置入牌堆底，或者牌堆牌数 +1 也为牌堆底

57.2 HpChangedData

`class HpChangedData`

num: integer
体力变化量，可能是正数或者负数

shield_lost: integer or nil

reason: str
体力变化原因

skillName: str
引起体力变化的技能名

damageEvent: None
引起这次体力变化的伤害数据

preventDying: None
是否阻止本次体力变更流程引发濒死流程

57.3 HpLostData

`class HpLostData`

num: integer
失去体力的数值

skillName: str
导致这次失去的技能名

57.4 MaxHpChangedData

`class MaxHpChangedData`

描述和一次体力变化有关的数据

描述跟失去体力有关的数据描述跟体力上限变化有关的数据

num: integer
体力上限变化量，可能是正数或者负数

57.5 DamageStruct

`class DamageStruct`

from: None
伤害来源

to: ServerPlayer
伤害目标

damage: integer
伤害值

card: None
造成伤害的牌

chain: None
伤害是否是铁索传导的伤害

damageType: None
伤害的属性

skillName: None
造成本次伤害的技能名

beginnerOfTheDamage: None
是否是本次铁索传导的起点

57.6 RecoverStruct

`class RecoverStruct`

who: ServerPlayer
回复体力的角色

num: integer
回复值

recoverBy: None
此次回复的回复来源

skillName: None
因何种技能而回复

card: None
造成此次回复的卡牌

57.7 DyingStruct

`class DyingStruct`

who: integer

damage: DamageStruct

ignoreDeath: None

57.8 DeathStruct

`class DeathStruct`

who: integer

damage: DamageStruct

57.9 UseExtraData

`class UseExtraData`

must_targets: None
必须选择这些目标?

include_targets: None
必须选其中一个目标?

exclusive_targets: None
只能选择这些目标?

bypass_distances: None
无距离限制?

bypass_times: None
无次数限制?

playing: None
(AI 专用) 出牌阶段?

57.10 CardUseStruct

`class CardUseStruct`

`from:` integer

`tos:` TargetGroup

`card:` Card

`toCard:` None

`responseToEvent:` None

`nullifiedTargets:` None

`extraUse:` None

`disresponsiveList:` None

`unoffsetableList:` None

`additionalDamage:` None

`additionalRecover:` None

`customFrom:` None

`cardsResponded:` None

`prohibitedCardNames:` None

`damageDealt:` None

`additionalEffect:` None

57.11 AimStruct

`class AimStruct`

`from:` integer

`card:` Card

`tos:` AimGroup

`to:` integer

`subTargets:` None

`targetGroup:` None

nullifiedTargets: None

firstTarget: boolean

additionalDamage: None

additionalRecover: None

disresponsive: None

unoffsetableList: None

additionalResponseTimes: None

fixedAddTimesResponsors: None

additionalEffect: None

57.12 CardEffectEvent

class CardEffectEvent

57.13 SkillEffectEvent

class SkillEffectEvent

from: integer

tos: list[integer]

cards: list[integer]

57.14 JudgeStruct

class JudgeStruct

who: ServerPlayer

card: Card

reason: str

pattern: str

skipDrop: None

57.15 CardResponseEvent

`class CardResponseEvent`

from: integer

card: Card

responseToEvent: None

skipDrop: None

customFrom: None

57.16 AskForCardUse

`class AskForCardUse`

user: ServerPlayer

cardName: str

pattern: str

result: CardUseStruct

57.17 AskForCardResponse

`class AskForCardResponse`

user: ServerPlayer

cardName: str

pattern: str

result: Card

57.18 PindianStruct

`class PindianStruct`

from: ServerPlayer

tos: list[ServerPlayer]

fromCard: Card

results: dict[integer, PindianResult]

reason: str

57.19 LogMessage

class LogMessage

type: str

log 主体

from: None

要替换%from 的玩家的 id

to: None

要替换%to 的玩家 id 列表

card: None

要替换%card 的卡牌 id 列表

arg: None

要替换%arg 的内容

arg2: None

要替换%arg2 的内容

arg3: None

要替换%arg3 的内容

toast: None

是否顺手把消息发送一条相同的 toast

57.20 DrawCardStruct

class DrawCardStruct

DamageStruct 用来描述和伤害事件有关的数据。

用来描述和回复体力有关的数据。askForUseCard 中的 extra_data

who: ServerPlayer

num: number

skillName: str

索引

A

abortPlayerArea() (*Room method*), 269
aboutToEffect() (*ActiveSkill method*), 206
AbstractRoom (*built-in class*), 222
AbstractRoom.alive_players (*attribute*), 222
AbstractRoom.banners (*attribute*), 222
AbstractRoom.card_marks (*attribute*), 222
AbstractRoom.current (*attribute*), 222
AbstractRoom.filtered_cards (*attribute*), 222
AbstractRoom.observers (*attribute*), 222
AbstractRoom.printed_cards (*attribute*), 222
AbstractRoom.skill_costs (*attribute*), 222
AbstractRoom.status_skills (*attribute*), 222
action() (*GameLogic method*), 278
ActiveSkill (*built-in class*), 203
ActiveSkill.card_num (*attribute*), 204
ActiveSkill.card_num_table (*attribute*), 204
ActiveSkill.interaction (*attribute*), 204
ActiveSkill.max_card_num (*attribute*), 204
ActiveSkill.max_target_num (*attribute*), 203
ActiveSkill.min_card_num (*attribute*), 204
ActiveSkill.min_target_num (*attribute*), 203
ActiveSkill.target_num (*attribute*), 203
ActiveSkill.target_num_table (*attribute*), 204
addAttachedKingdom() (*Skill method*), 202
addBuddy() (*Player method*), 234
addBuddy() (*ServerPlayer method*), 276
addCard() (*Engine method*), 187
addCard() (*Package method*), 220
addCardMark() (*Room method*), 244
addCards() (*Engine method*), 187
addCards() (*Package method*), 220
addCards() (*Player method*), 227
addCardUseHistory() (*Player method*), 230
addCardUseHistory() (*ServerPlayer method*), 274
addCleaner() (*GameEvent method*), 282
addCompanions() (*General method*), 218
addExitFunc() (*GameEvent method*), 282
addFakeSkill() (*ServerPlayer method*), 275
addGameEvent() (*Engine method*), 188
addGameMode() (*Engine method*), 188
addGameMode() (*Package method*), 220
addGameModeFilter() (*Package method*), 220
addGameModes() (*Engine method*), 188
addGeneral() (*Engine method*), 187
addGeneral() (*Package method*), 220
addGenerals() (*Engine method*), 187
addMark() (*Card method*), 194
addMark() (*Player method*), 226
addMiniGame() (*Engine method*), 188
addPlayerMark() (*Room method*), 243
addPoxiMethod() (*Engine method*), 188
addQmlMark() (*Engine method*), 188
addRelatedSkill() (*General method*), 217
addRelatedSkill() (*Skill method*), 202

addSkill() (*Engine method*), 186
addSkill() (*General method*), 217
addSkill() (*Player method*), 232
addSkills() (*Engine method*), 186
addSkillUseHistory() (*Player method*), 230
addSkillUseHistory() (*ServerPlayer method*), 275
addSubcard() (*Card method*), 193
addSubcards() (*Card method*), 193
addToPile() (*ServerPlayer method*), 274
addTriggerSkill() (*GameLogic method*), 278
addVirtualEquip() (*Player method*), 227
addVirtualEquip() (*ServerPlayer method*), 274
adjustSeats() (*Room method*), 267
afterUse() (*ViewAsSkill method*), 207
AimStruct (*built-in class*), 288
AimStruct.additionalDamage (*attribute*), 289
AimStruct.additionalEffect (*attribute*), 289
AimStruct.additionalRecover (*attribute*), 289
AimStruct.additionalResponseTimes (*attribute*), 289
AimStruct.card (*attribute*), 288
AimStruct.disresponsive (*attribute*), 289
AimStruct.firstTarget (*attribute*), 289
AimStruct.fixedAddTimesResponsors (*attribute*), 289
AimStruct.from (*attribute*), 288
AimStruct.nullifiedTargets (*attribute*), 288
AimStruct.subTargets (*attribute*), 288
AimStruct.targetGroup (*attribute*), 288
AimStruct.to (*attribute*), 288
AimStruct.tos (*attribute*), 288
AimStruct.unoffsetableList (*attribute*), 289
Armor (*built-in class*), 199
askForAddTarget() (*Room method*), 257
askForAG() (*Room method*), 260
askForCard() (*Room method*), 251
askForCardChosen() (*Room method*), 255
AskForCardResponse (*built-in class*), 290
AskForCardResponse.cardName (*attribute*), 290
AskForCardResponse.pattern (*attribute*), 290
AskForCardResponse.result (*attribute*), 290
AskForCardResponse.user (*attribute*), 290
askForCardsChosen() (*Room method*), 255
AskForCardUse (*built-in class*), 290
AskForCardUse.cardName (*attribute*), 290
AskForCardUse.pattern (*attribute*), 290
AskForCardUse.result (*attribute*), 290
AskForCardUse.user (*attribute*), 290
askForChoice() (*Room method*), 256
askForChoices() (*Room method*), 256
askForChooseCardAndPlayers() (*Room method*), 252
askForChooseCardsAndPlayers() (*Room method*), 252
askForChooseKingdom() (*Room method*), 255
askForChoosePlayers() (*Room method*), 251
askForChooseToMoveCardInBoard() (*Room method*), 262
askForCustomDialog() (*Room method*), 261
askForDiscard() (*Room method*), 250
askForExchange() (*Room method*), 258
askForGeneral() (*Room method*), 254
askForGuanxing() (*Room method*), 258
askForMiniGame() (*Room method*), 261
askForMoveCardInBoard() (*Room method*), 261
askForNullification() (*Room method*), 260
askForPoxi() (*Room method*), 256
askForResponse() (*Room method*), 259
askForSkillInvoke() (*Room method*), 257
askForUseActiveSkill() (*Room method*), 250
askForUseCard() (*Room method*), 259
askForYiji() (*Room method*), 253
assignRoles() (*GameLogic method*), 277
attachSkillToPlayers() (*GameLogic method*), 278
AttackRangeSkill (*built-in class*), 211

B

BasicCard (*built-in class*), 197
beforeUse() (*ViewAsSkill method*), 207
breakEvent() (*GameLogic method*), 280
breakTurn() (*GameLogic method*), 280
broadcastGeneral() (*GameLogic method*), 278
broadcastPlaySound() (*Room method*), 249
broadcastProperty() (*Room method*), 246
broadcastSkillInvoke() (*Room method*), 249
broadcastSkillInvoke() (*ServerPlayer method*), 275
buildPlayerCircle() (*GameLogic method*), 278
bury() (*ServerPlayer method*), 274
bypassDistancesCheck() (*TargetModSkill method*), 215
bypassTimesCheck() (*TargetModSkill method*), 214

C

canMoveCardInBoard() (*Room method*), 269
canMoveCardInBoardTo() (*Player method*), 234
canMoveCardIntoEquip() (*Player method*), 233
canMoveCardsInBoardTo() (*Player method*), 234
canPindian() (*Player method*), 233
canRefresh() (*TriggerSkill method*), 208
canUse() (*ActiveSkill method*), 204
canUse() (*Player method*), 232
canUseGeneral() (*Engine method*), 187
canUseTo() (*Player method*), 232
canWake() (*TriggerSkill method*), 210
Card (*built-in class*), 191
Card.area (*attribute*), 192
Card.color (*attribute*), 191
CardEffectEvent (*built-in class*), 289
cardFilter() (*ActiveSkill method*), 204
cardFilter() (*FilterSkill method*), 214
cardFilter() (*ViewAsSkill method*), 207
Card.id (*attribute*), 191
Card.is_damage_card (*attribute*), 192
Card.is_derived (*attribute*), 192
Card.is_passive (*attribute*), 192
Card.mark (*attribute*), 192
Card.multiple_targets (*attribute*), 192
Card.name (*attribute*), 191
Card.number (*attribute*), 191
Card.package (*attribute*), 191
CardResponseEvent (*built-in class*), 290
CardResponseEvent.card (*attribute*), 290
CardResponseEvent.customFrom (*attribute*), 290
CardResponseEvent.from (*attribute*), 290
CardResponseEvent.responseToEvent (*attribute*), 290
CardResponseEvent.skipDrop (*attribute*), 290
Card.skill (*attribute*), 192
Card.skillName (*attribute*), 192
Card.skillNames (*attribute*), 192
CardsMoveStruct (*built-in class*), 284
CardsMoveStruct.drawPilePosition (*attribute*), 284
CardsMoveStruct.from (*attribute*), 284
CardsMoveStruct.moveInfo (*attribute*), 284
CardsMoveStruct.moveReason (*attribute*), 284
CardsMoveStruct.moveVisible (*attribute*), 284
CardsMoveStruct.proposer (*attribute*), 284
CardsMoveStruct.skillName (*attribute*), 284
CardsMoveStruct.specialName (*attribute*), 284
CardsMoveStruct.specialVisible (*attribute*), 284
CardsMoveStruct.to (*attribute*), 284
CardsMoveStruct.toArea (*attribute*), 284
Card.special_skills (*attribute*), 192
Card.sub_type (*attribute*), 191
Card.subcards (*attribute*), 192
Card.suit (*attribute*), 191
Card.trueName (*attribute*), 191
Card.type (*attribute*), 191
CardUseStruct (*built-in class*), 288
CardUseStruct.additionalDamage (*attribute*), 288

CardUseStruct.additionalEffect (*attribute*), 288
CardUseStruct.additionalRecover (*attribute*), 288
CardUseStruct.card (*attribute*), 288
CardUseStruct.cardsResponded (*attribute*), 288
CardUseStruct.customFrom (*attribute*), 288
CardUseStruct.damageDealt (*attribute*), 288
CardUseStruct.disresponsiveList (*attribute*), 288
CardUseStruct.extraUse (*attribute*), 288
CardUseStruct.from (*attribute*), 288
CardUseStruct.nullifiedTargets (*attribute*), 288
CardUseStruct.prohibitedCardNames (*attribute*), 288
CardUseStruct.responseToEvent (*attribute*), 288
CardUseStruct.toCard (*attribute*), 288
CardUseStruct.tos (*attribute*), 288
CardUseStruct.unoffsetableList (*attribute*), 288
changeHero() (*Room method*), 245
changeHp() (*Room method*), 265
changeKingdom() (*Room method*), 245
changeMaxHp() (*Room method*), 265
changePhase() (*ServerPlayer method*), 273
changeShield() (*Room method*), 265
checkNoHuman() (*Room method*), 241
chooseGenerals() (*GameLogic method*), 277
clearEvent() (*GameLogic method*), 278
clearFlags() (*Player method*), 226
clearPiles() (*ServerPlayer method*), 274
clearSubcards() (*Card method*), 193
clone() (*Card method*), 192
clone() (*EquipCard method*), 198
clone() (*Weapon method*), 198
cloneCard() (*Engine method*), 187
closeAG() (*Room method*), 261
compareColorWith() (*Card method*), 196
compareGenderWith() (*Player method*), 235
compareNumberWith() (*Card method*), 196
compareSuitWith() (*Card method*), 195
control() (*ServerPlayer method*), 276
cost() (*TriggerSkill method*), 209
countInFunc() (*GameMode method*), 237
currentRoom() (*Engine method*), 190

D

damage() (*Room method*), 266
DamageStruct (*built-in class*), 286
DamageStruct.beginnerOfTheDamage (*attribute*), 286
DamageStruct.card (*attribute*), 286
DamageStruct.chain (*attribute*), 286
DamageStruct.damage (*attribute*), 286
DamageStruct.damageType (*attribute*), 286
DamageStruct.from (*attribute*), 286
DamageStruct.skillName (*attribute*), 286
DamageStruct.to (*attribute*), 286
deadPlayerFilter() (*Room method*), 242
DeathStruct (*built-in class*), 287
DeathStruct.damage (*attribute*), 287
DeathStruct.who (*attribute*), 287
DefensiveRide (*built-in class*), 199
delay() (*Room method*), 247
DelayedTrickCard (*built-in class*), 197
DistanceSkill (*built-in class*), 211
distanceTo() (*Player method*), 229
doAnimate() (*Room method*), 248
doBroadcastNotify() (*Room method*), 246
doBroadcastRequest() (*Room method*), 247
doCardEffect() (*Room method*), 262
doCardUseEffect() (*Room method*), 262
doCost() (*TriggerSkill method*), 209
doIndicate() (*Room method*), 249
doNotify() (*ServerPlayer method*), 272
doRaceRequest() (*Room method*), 247
doRequest() (*Room method*), 246
doRequest() (*ServerPlayer method*), 272
doSuperLightBox() (*Room method*), 249
doYiji() (*Room method*), 264
drawCards() (*Room method*), 264
drawCards() (*ServerPlayer method*), 273
DrawCardStruct (*built-in class*), 291
DrawCardStruct.num (*attribute*), 291
DrawCardStruct.skillName (*attribute*), 291
DrawCardStruct.who (*attribute*), 291

dumpAllEvents() (*GameLogic method*), 280
dumpEventStack() (*GameLogic method*), 280
DyingStruct (*built-in class*), 287
DyingStruct.damage (*attribute*), 287
DyingStruct.ignoreDeath (*attribute*), 287
DyingStruct.who (*attribute*), 287

E

enabledAtPlay() (*ViewAsSkill method*), 207
enabledAtResponse() (*ViewAsSkill method*), 207
enableToWake() (*TriggerSkill method*), 210
endPlayPhase() (*ServerPlayer method*), 273
Engine (*built-in class*), 184
Engine.all_card_types (*attribute*), 185
Engine.cards (*attribute*), 185
Engine.currentResponsePattern (*attribute*), 185
Engine.currentResponseReason (*attribute*), 185
Engine.extension_names (*attribute*), 184
Engine.extensions (*attribute*), 184
Engine.filtered_cards (*attribute*), 185
Engine.game_mode_disabled (*attribute*), 185
Engine.game_modes (*attribute*), 185
Engine.generals (*attribute*), 184
Engine.global_status_skill (*attribute*), 184
Engine.global_trigger (*attribute*), 184
Engine.lords (*attribute*), 185
Engine.mini_games (*attribute*), 185
Engine.package_names (*attribute*), 184
Engine.packages (*attribute*), 184
Engine.poxi_methods (*attribute*), 185
Engine.printed_cards (*attribute*), 185
Engine.qml_marks (*attribute*), 185
Engine.related_skills (*attribute*), 184
Engine.same_generals (*attribute*), 185
Engine.skills (*attribute*), 184
Engine.translations (*attribute*), 185
enterDying() (*Room method*), 266
EquipCard (*built-in class*), 198
EquipCard.equip_skill (*attribute*), 198
excludeFrom() (*MaxCardsSkill method*), 212
exec() (*GameEvent method*), 283
Exppattern (*built-in class*), 238
Exppattern.matchers (*attribute*), 238

F

feasible() (*ActiveSkill method*), 205
fillAG() (*Room method*), 260
filterCard() (*Engine method*), 190
filterHandcards() (*Player method*), 228
FilterSkill (*built-in class*), 214
findGeneral() (*Room method*), 254
findGenerals() (*Room method*), 254
findParent() (*GameEvent method*), 282

G

gainAnExtraPhase() (*ServerPlayer method*), 273
gainAnExtraTurn() (*ServerPlayer method*), 273
GameEvent (*built-in class*), 281
GameEvent.clear_func (*attribute*), 281
GameEvent.data (*attribute*), 281
GameEvent.end_id (*attribute*), 281
GameEvent.event (*attribute*), 281
GameEvent.exec_ret (*attribute*), 282
GameEvent.exit_func (*attribute*), 282
GameEvent.extra_clear_funcs (*attribute*), 281
GameEvent.extra_exit_funcs (*attribute*), 282
GameEvent.id (*attribute*), 281
GameEvent.interrupted (*attribute*), 282
GameEvent.killed (*attribute*), 282
GameEvent.main_func (*attribute*), 281
GameEvent.parent (*attribute*), 281
GameEvent.prepare_func (*attribute*), 281
GameEvent.room (*attribute*), 281
GameEvent.status (*attribute*), 282
GameLogic (*built-in class*), 277
GameLogic.all_game_events (*attribute*), 277
GameLogic.cleaner_stack (*attribute*), 277
GameLogic.current_event_id (*attribute*), 277
GameLogic.event_recorder (*attribute*), 277

GameLogic.game_event_stack (*attribute*), 277
GameLogic.refresh_skill_table (*attribute*), 277
GameLogic.role_table (*attribute*), 277
GameLogic.room (*attribute*), 277
GameLogic.skill_priority_table (*attribute*), 277
GameLogic.skill_table (*attribute*), 277
GameLogic.skills (*attribute*), 277
GameMode (*built-in class*), 236
GameMode.blacklist (*attribute*), 236
GameMode.logic (*attribute*), 236
GameMode.maxPlayer (*attribute*), 236
GameMode.minPlayer (*attribute*), 236
GameMode.name (*attribute*), 236
GameMode.rule (*attribute*), 236
GameMode.whitelist (*attribute*), 236
gameOver() (*Room method*), 268
General (*built-in class*), 216
General.companions (*attribute*), 217
General.deputyMaxHpAdjustedValue (*attribute*), 216
General.gender (*attribute*), 217
General.hidden (*attribute*), 217
General.hp (*attribute*), 216
General.kingdom (*attribute*), 216
General.mainMaxHpAdjustedValue (*attribute*), 216
General.maxHp (*attribute*), 216
General.name (*attribute*), 216
General.other_skills (*attribute*), 217
General.package (*attribute*), 216
General.related_other_skills (*attribute*), 217
General.related_skills (*attribute*), 217
General.shield (*attribute*), 217
General.skills (*attribute*), 217
General.subkingdom (*attribute*), 216
General.total_hidden (*attribute*), 217
General.trueName (*attribute*), 216
getActualDamageEvents() (*GameLogic method*), 279
getAlivePlayers() (*Room method*), 242
getAllCardIds() (*Engine method*), 189
getAllGenerals() (*Engine method*), 189
getAllPlayers() (*Room method*), 242
getAllSkills() (*Player method*), 232
getAttackRange() (*Player method*), 229
getAvailableEquipSlots() (*Player method*), 234
getBanner() (*AbstractRoom method*), 223
getCardArea() (*AbstractRoom method*), 223
getCardArea() (*Room method*), 241
getCardById() (*Engine method*), 189
getCardIds() (*Player method*), 228
getCardOwner() (*Room method*), 241
getCardsFromPileByRule() (*Room method*), 268
getColorString() (*Card method*), 194
getCorrect() (*AttackRangeSkill method*), 211
getCorrect() (*DistanceSkill method*), 211
getCorrect() (*MaxCardsSkill method*), 212
getCurrentCleaner() (*GameLogic method*), 278
getCurrentEvent() (*GameLogic method*), 278
getCurrentExtraTurnReason() (*ServerPlayer method*), 273
getCurrentSkillName() (*GameLogic method*), 279
getDescription() (*Engine method*), 190
getDistanceLimit() (*ActiveSkill method*), 205
getDistanceLimit() (*TargetModSkill method*), 215
getEffectiveId() (*Card method*), 193
getEquipment() (*Player method*), 229
getEquipments() (*Player method*), 229
getEventsByRule() (*GameLogic method*), 279
getEventsOfScope() (*GameLogic method*), 279
getExtraTargetNum() (*TargetModSkill method*), 215
getFixed() (*AttackRangeSkill method*), 211
getFixed() (*DistanceSkill method*), 211
getFixed() (*MaxCardsSkill method*), 212
getGeneralMaxHp() (*Player method*), 226
getGeneralsRandomly() (*Engine method*), 188
getHandcardNum() (*Player method*), 228

getHandlyIds() (*Player method*), 228
getIdList() (*Card method*), 196
getInvalidity() (*InvaliditySkill method*), 214
getLastAlive() (*Player method*), 230
getLord() (*Room method*), 242
getLostHp() (*Player method*), 231
getMark() (*Card method*), 195
getMark() (*Player method*), 227
getMarkNames() (*Card method*), 195
getMarkNames() (*Player method*), 227
getMaxCardNum() (*ActiveSkill method*), 205
getMaxCards() (*Player method*), 229
getMaxTargetNum() (*ActiveSkill method*), 205
getMaxUseTime() (*UsableSkill method*), 203
getMinCardNum() (*ActiveSkill method*), 205
getMinTargetNum() (*ActiveSkill method*), 205
getMostRecentEvent() (*GameLogic method*), 279
getNCards() (*Room method*), 243
getNextAlive() (*Player method*), 230
getNGenerals() (*Room method*), 253
getNumberStr() (*Card method*), 194
getOtherPlayers() (*Room method*), 242
getPile() (*Player method*), 228
getPileNameOfId() (*Player method*), 228
getPlayerById() (*AbstractRoom method*), 222
getPlayerById() (*Room method*), 241
getQuestSkillState() (*Player method*), 234
getResidueNum() (*TargetModSkill method*), 214
getSameGenerals() (*Engine method*), 187
getSkillNameList() (*General method*), 218
getSkills() (*Package method*), 220
getSubcardsByRule() (*Room method*), 268
getSubtypeString() (*Card method*), 194
getSuitCompletedString() (*Card method*), 194
getSuitString() (*Card method*), 193
getSwitchSkillState() (*Player method*), 233
getTag() (*Room method*), 245
getTypeString() (*Card method*), 194
getUseExtraTargets() (*Room method*), 257
getVirualEquip() (*Player method*), 227
getWinner() (*GameMode method*), 237

H

handleAddLoseSkills() (*Room method*), 266
handleCardEffect() (*Room method*), 263
handleUseCardReply() (*Room method*), 259
hasDelayedTrick() (*Player method*), 228
hasEmptyEquipSlot() (*Player method*), 234
hasFlag() (*Player method*), 226
hasSkill() (*Player method*), 231
hideGeneral() (*ServerPlayer method*), 276
HpChangedData (*built-in class*), 285
HpChangedData.damageEvent (*attribute*), 285
HpChangedData.num (*attribute*), 285
HpChangedData.preventDying (*attribute*), 285
HpChangedData.reason (*attribute*), 285
HpChangedData.shield_lost (*attribute*), 285
HpChangedData.skillName (*attribute*), 285
HpLostData (*built-in class*), 285
HpLostData.num (*attribute*), 285
HpLostData.skillName (*attribute*), 285

I

initialize() (*AbstractRoom method*), 222
initialize() (*ActiveSkill method*), 204
initialize() (*Armor method*), 199
initialize() (*BasicCard method*), 197
initialize() (*Card method*), 192
initialize() (*DefensiveRide method*), 199
initialize() (*DelayedTrickCard method*), 197
initialize() (*Engine method*), 185
initialize() (*EquipCard method*), 198
initialize() (*Exppattern method*), 238

initialize() (*GameEvent method*), 282
initialize() (*GameLogic method*), 277
initialize() (*GameMode method*), 236
initialize() (*General method*), 217
initialize() (*OffensiveRide method*), 199
initialize() (*Package method*), 220
initialize() (*Player method*), 226
initialize() (*Room method*), 240
initialize() (*ServerPlayer method*), 272
initialize() (*Skill method*), 202
initialize() (*StatusSkill method*), 211
initialize() (*Treasure method*), 200
initialize() (*TrickCard method*), 197
initialize() (*TriggerSkill method*), 208
initialize() (*UsableSkill method*), 203
initialize() (*ViewAsSkill method*), 207
initialize() (*Weapon method*), 198
inMyAttackRange() (*Player method*), 230
insideExtraTurn() (*ServerPlayer method*), 273
InvaliditySkill (*built-in class*), 214
isAlive() (*ServerPlayer method*), 272
isAllNude() (*Player method*), 231
isBuddy() (*Player method*), 234
isCommonTrick() (*Card method*), 194
isEffectable() (*Skill method*), 202
isEquipmentSkill() (*Skill method*), 202
isFakeSkill() (*ServerPlayer method*), 276
isKongcheng() (*Player method*), 231
isNude() (*Player method*), 231
isProhibited() (*Player method*), 233
isProhibited() (*ProhibitSkill method*), 212
isReady() (*Room method*), 240
isRemoved() (*Player method*), 229
isSwitchSkill() (*Skill method*), 202
isVirtual() (*Card method*), 193
isWounded() (*Player method*), 231

J

judge() (*Room method*), 267
JudgeStruct (*built-in class*), 289
JudgeStruct.card (*attribute*), 289
JudgeStruct.pattern (*attribute*), 289
JudgeStruct.reason (*attribute*), 289
JudgeStruct.skipDrop (*attribute*), 289
JudgeStruct.who (*attribute*), 289

K

killPlayer() (*Room method*), 266

L

loadDisabled() (*Engine method*), 186
loadPackage() (*Engine method*), 185
loadPackages() (*Engine method*), 186
loadTranslationTable() (*Engine method*), 186
LogMessage (*built-in class*), 291
LogMessage.arg (*attribute*), 291
LogMessage.arg2 (*attribute*), 291
LogMessage.arg3 (*attribute*), 291
LogMessage.card (*attribute*), 291
LogMessage.from (*attribute*), 291
LogMessage.to (*attribute*), 291
LogMessage.toast (*attribute*), 291
LogMessage.type (*attribute*), 291
loseFakeSkill() (*ServerPlayer method*), 275
loseHp() (*Room method*), 265
loseSkill() (*Player method*), 232

M

makeGeneralPile() (*Room method*), 240
marshal() (*ServerPlayer method*), 272
match() (*Exppattern method*), 238
matchExp() (*Exppattern method*), 238
matchPattern() (*Card method*), 193
MaxCardsSkill (*built-in class*), 212
MaxHpChangedData (*built-in class*), 285
MaxHpChangedData.num (*attribute*), 285
modTargetFilter() (*ActiveSkill method*), 204
moveCardIntoEquip() (*Room method*), 265
moveCards() (*Room method*), 263
moveCardTo() (*Room method*), 264

N

notifyMoveCards() (*Room method*), 247
notifyMoveFocus() (*Room method*), 247
notifyProperty() (*Room method*), 246
notifySkillInvoked() (*Room method*), 249

O

obtainCard() (*Room method*), 263

OffensiveRide (*built-in class*), 199
onAction() (*ActiveSkill method*), 206
onEffect() (*ActiveSkill method*), 206
onInstall() (*EquipCard method*), 198
onNullified() (*ActiveSkill method*), 206
onUninstall() (*EquipCard method*), 198
onUse() (*ActiveSkill method*), 206

P

Package (*built-in class*), 219
Package.cards (*attribute*), 219
Package.extensionName (*attribute*), 219
Package.extra_skills (*attribute*), 219
Package.game_modes (*attribute*), 219
Package.game_modes_blacklist (*attribute*), 219
Package.game_modes_whitelist (*attribute*), 219
Package.generals (*attribute*), 219
Package.name (*attribute*), 219
Package.related_skills (*attribute*), 219
Package.type (*attribute*), 219
Parse() (*Exppattern method*), 238
pindian() (*Room method*), 267
pindian() (*ServerPlayer method*), 275
PindianStruct (*built-in class*), 290
PindianStruct.from (*attribute*), 290
PindianStruct.fromCard (*attribute*), 290
PindianStruct.reason (*attribute*), 291
PindianStruct.results (*attribute*), 291
PindianStruct.tos (*attribute*), 290
play() (*ServerPlayer method*), 273
Player (*built-in class*), 224
Player.buddy_list (*attribute*), 226
Player.cardUsedHistory (*attribute*), 225
Player.chained (*attribute*), 225
Player.dead (*attribute*), 225
Player.deputyGeneral (*attribute*), 224
Player.derivative_skills (*attribute*), 225
Player.dying (*attribute*), 225
Player.faceup (*attribute*), 225
Player.fixedDistance (*attribute*), 225
Player.flag (*attribute*), 225
Player.gender (*attribute*), 224
Player.general (*attribute*), 224
Player.hp (*attribute*), 224
Player.id (*attribute*), 224
Player.kingdom (*attribute*), 224
Player.mark (*attribute*), 225
Player.maxHp (*attribute*), 224
Player.next (*attribute*), 225
Player.phase (*attribute*), 225
Player.player_cards (*attribute*), 225
Player.player_skills (*attribute*), 225
Player.role (*attribute*), 224
Player.seat (*attribute*), 225
Player.shield (*attribute*), 224
Player.skillUsedHistory (*attribute*), 225
Player.special_cards (*attribute*), 225
Player.tag (*attribute*), 225
Player.virtual_equips (*attribute*), 225
prelightSkill() (*ServerPlayer method*), 276
prepareDrawPile() (*GameLogic method*), 278
prepareForStart() (*GameLogic method*), 278
prependExitFunc() (*GameEvent method*), 282
printCard() (*Room method*), 269
prohibitDiscard() (*Player method*), 233
prohibitDiscard() (*ProhibitSkill method*), 213
prohibitPindian() (*ProhibitSkill method*), 213
prohibitResponse() (*Player method*), 233
prohibitResponse() (*ProhibitSkill method*), 213
prohibitReveal() (*Player method*), 233
ProhibitSkill (*built-in class*), 212
prohibitUse() (*Player method*), 233
prohibitUse() (*ProhibitSkill method*), 213
prompt() (*ActiveSkill method*), 206
prompt() (*ViewAsSkill method*), 208
pushEvent() (*GameLogic method*), 278

R

recastCard() (*Room method*), 267
reconnect() (*ServerPlayer method*), 272
recover() (*Room method*), 266
RecoverStruct (*built-in class*), 286
RecoverStruct.card (*attribute*), 286
RecoverStruct.num (*attribute*), 286
RecoverStruct.recoverBy (*attribute*), 286
RecoverStruct.skillName (*attribute*), 286

RecoverStruct.who (*attribute*), 286
refresh() (*TriggerSkill method*), 208
removeBuddy() (*Player method*), 234
removeBuddy() (*ServerPlayer method*), 276
removeCardMark() (*Room method*), 244
removeCards() (*Player method*), 227
removeFixedDistance() (*Player method*), 229
removeMark() (*Card method*), 195
removeMark() (*Player method*), 226
removePlayerMark() (*Room method*), 243
removeTag() (*Room method*), 245
removeVirtualEquip() (*Player method*), 227
removeVirtualEquip() (*ServerPlayer method*), 274
reset() (*ServerPlayer method*), 275
responseCard() (*Room method*), 263
resume() (*Room method*), 240
resumeEvent() (*GameLogic method*), 278
resumePlayerArea() (*Room method*), 269
retrial() (*Room method*), 267
returnToGeneralPile() (*Room method*), 254
revealBySkillName() (*ServerPlayer method*), 276
revealGeneral() (*ServerPlayer method*), 276
revealGenerals() (*ServerPlayer method*), 276
revivePlayer() (*Room method*), 268
Room (*built-in class*), 239
Room.alive_players (*attribute*), 239
Room.card_marks (*attribute*), 240
Room.card_place (*attribute*), 240
Room.current (*attribute*), 239
Room.discard_pile (*attribute*), 240
Room.draw_pile (*attribute*), 240
Room.game_finished (*attribute*), 239
Room.game_started (*attribute*), 239
Room.general_pile (*attribute*), 240
Room.id (*attribute*), 239
Room.logic (*attribute*), 240
Room.observers (*attribute*), 239
Room.owner_map (*attribute*), 240
Room.players (*attribute*), 239
Room.processing_area (*attribute*), 240
Room.request_queue (*attribute*), 240
Room.request_self (*attribute*), 240
Room.room (*attribute*), 239
Room.settings (*attribute*), 240
Room.skill_costs (*attribute*), 240
Room.tag (*attribute*), 240
Room.timeout (*attribute*), 239
Room.void (*attribute*), 240
run() (*GameLogic method*), 277
run() (*Room method*), 241

S

searchEvents() (*GameEvent method*), 282
sendCardVirtName() (*Room method*), 248
sendFootnote() (*Room method*), 248
sendLog() (*Room method*), 248
sendLogEvent() (*Room method*), 249
ServerPlayer (*built-in class*), 271
ServerPlayer.ai (*attribute*), 271
ServerPlayer.ai_data (*attribute*), 271
ServerPlayer.client_reply (*attribute*), 271
ServerPlayer.default_reply (*attribute*), 271
ServerPlayer.mini_game_data (*attribute*), 271
ServerPlayer.next (*attribute*), 271
ServerPlayer.phase_index (*attribute*), 271
ServerPlayer.phase_state (*attribute*), 271
ServerPlayer.phases (*attribute*), 271
ServerPlayer.prelighted_skills (*attribute*), 271
ServerPlayer.reply_cancel (*attribute*), 271
ServerPlayer.reply_ready (*attribute*), 271
ServerPlayer.request_data (*attribute*), 271
ServerPlayer.role_shown (*attribute*), 271
ServerPlayer.room (*attribute*), 271
ServerPlayer.serverplayer (*attribute*), 271
ServerPlayer.skipped_phases (*attribute*), 271
setBanner() (*AbstractRoom method*), 223
setBanner() (*Room method*), 245
setCardArea() (*Room method*), 241

setCardEmotion() (*Room method*), 248
setCardMark() (*Room method*), 244
setCardUseHistory() (*Player method*), 230
setCardUseHistory() (*ServerPlayer method*), 274
setChainState() (*ServerPlayer method*), 275
setDeputyGeneral() (*Room method*), 245
setEmotion() (*Room method*), 248
setFixedDistance() (*Player method*), 229
setFlag() (*Player method*), 226
setGameModeFilter() (*Package method*), 220
setGeneral() (*Player method*), 226
setMark() (*Card method*), 195
setMark() (*Player method*), 226
setPlayerGeneral() (*Room method*), 245
setPlayerMark() (*Room method*), 243
setPlayerProperty() (*Room method*), 244
setPlayerRest() (*Room method*), 270
setSkillUseHistory() (*Player method*), 231
setSkillUseHistory() (*ServerPlayer method*), 275
setTag() (*Room method*), 244
showCards() (*ServerPlayer method*), 272
shuffleDrawPile() (*Room method*), 268
shutdown() (*GameEvent method*), 283
Skill (*built-in class*), 201
Skill.anim_type (*attribute*), 201
Skill.attached_equip (*attribute*), 201
SkillEffectEvent (*built-in class*), 289
SkillEffectEvent.cards (*attribute*), 289
SkillEffectEvent.from (*attribute*), 289
SkillEffectEvent.tos (*attribute*), 289
Skill.frequency (*attribute*), 201
Skill.global (*attribute*), 201
Skill.mute (*attribute*), 201
Skill.name (*attribute*), 201
Skill.package (*attribute*), 201
Skill.relate_to_place (*attribute*), 202
Skill.related_skills (*attribute*), 201
Skill.switchSkillName (*attribute*), 202
Skill.trueName (*attribute*), 201
Skill.visible (*attribute*), 201
skip() (*ServerPlayer method*), 273
sortPlayersByAction() (*Room method*), 242
start() (*GameLogic method*), 278
StatusSkill (*built-in class*), 211
StatusSkill.global (*attribute*), 211
strToNumber() (*Card method*), 194
surrenderFunc() (*GameMode method*), 237
swapSeat() (*Room method*), 268

T

takeAG() (*Room method*), 261
targetFilter() (*ActiveSkill method*), 204
TargetModSkill (*built-in class*), 214
throwAllCards() (*ServerPlayer method*), 274
throwAllMarks() (*ServerPlayer method*), 274
throwCard() (*Room method*), 267
toLogString() (*Card method*), 196
translate() (*Engine method*), 186
translate() (*GameEvent method*), 282
Treasure (*built-in class*), 200
TrickCard (*built-in class*), 197
trigger() (*GameLogic method*), 278
trigger() (*TriggerSkill method*), 209
triggerable() (*TriggerSkill method*), 209
TriggerSkill (*built-in class*), 208
TriggerSkill.events (*attribute*), 208
TriggerSkill.global (*attribute*), 208
TriggerSkill.priority_table (*attribute*), 208
TriggerSkill.refresh_events (*attribute*), 208
turnOver() (*ServerPlayer method*), 272

U

updateQuestSkillState() (*Room method*), 269
UsableSkill (*built-in class*), 203
UsableSkill.derived_piles (*attribute*), 203
UsableSkill.expand_pile (*attribute*), 203
UsableSkill.main_skill (*attribute*), 203
UsableSkill.max_use_time (*attribute*), 203
use() (*TriggerSkill method*), 210
useCard() (*Room method*), 262